
Automotive SoC Motor Driver

1.0 ADVANCED MOTOR CONTROL TIMER (AMCT)

1.1 Features

- bemf comparators degauss and debounce filters
- phase shift generation for hall sensors in case of mechanical misposition
- debounce filters for hall sensors
- dedicated timer to calculate hall sensor frequency
- commutation section report to correctly energize motor phases
- automatic phase-disable signal generation for pulse width modulation unit to open window and sample bemf information
- utilize automatic phase-disable signal generation to implement trapezoidal control for high speed motors
- dedicated counter which can be programmed to report motor frequency and disables phases automatically to open bemf window at a correct time for systems with bemf based control algorithm
- dedicated hardware which reports bemf angle zero crossing error with respect to window-opening position
- commutation section report to energize correct motor phases
- dedicated hardware to report current polarity in each phase of the three-phase system
- dedicated hardware to sample bemf comparators information which helps with motor initial position detection and windmilling algorithm

1.2 Introduction

The AMCT module incorporates major functions required to achieve robust start-up algorithms such as initial position detection and windmilling. In addition, AMCT modules has dedicated hardware for motor control algorithms which are based on hall sensors. Among the features are hall sensor output frequency report, debounce filtering to remove noise from hall sensors outputs and generate phase shift to hall sensor output in case of mechanical misadjustments or requirement to generate phase advance and delay. AMCT module also has dedicated hardware for application which are based on trapezoidal sensor-less bemf based system. A programmable dedicated counter is used to report motor frequency. From this counter motor commutation section information are generated in order energize correct motor phases and generate a phase disable signal automatically to open motor phase and sample bemf information. A dedicated hardware will also report bemf zero-crossing error with respect to window opening position in order to correct the counter value in the following motor control cycles. In addition, there are dedicated debounce and degaus filters to remove the noise from bemf signals for applications with sensor-less bemf based control.

There are multiple trigger signals generated from AMCT module which are processed in the interrupt management system and send to the NVIC to generate interrupt and receive the desired information as soon as possible. More information about interrupts and features included in the AMCT module are provided in the following sections.

Automotive SoC Motor Driver

1.3 Functional Description

The top-level architecture of the AMCT module is shown in Figure 1-1. The hall sensor inputs can be connected to GPIO pins PD0-PD7 and through registers settings the desired GPIO pins to which hall sensor is connected can be selected. The motor phases are connected to Sx nodes and are then processed by bemf and phase comparators. The desired signal, phase comparator or bemf comparator outputs, must be selected by the settings in the gate driver unit and the final output is then available in the AMCT module after selection. The phase comparator output is used in the current polarity detection (CPD) subsystem. The bemf comparator output is used in initial position detection (IPD) and bemf based system. The control logic signals from pulse width modulation unit (PWMU) is also available in the AMCT module to sample the bemf and phase comparator information at the correct timing which are used in initial position detection (IPD) and CPD subsystems. Three Phasex_Dis signals are generated from AMCT module to PWMU in order to disable three motor phases as commanded. The bemf based system contains degauss and debounce filters for the bemf comparators as well dedicated counter and bemf zero crossing error measurement hardware to facilitate sensor-less bemf motor control.

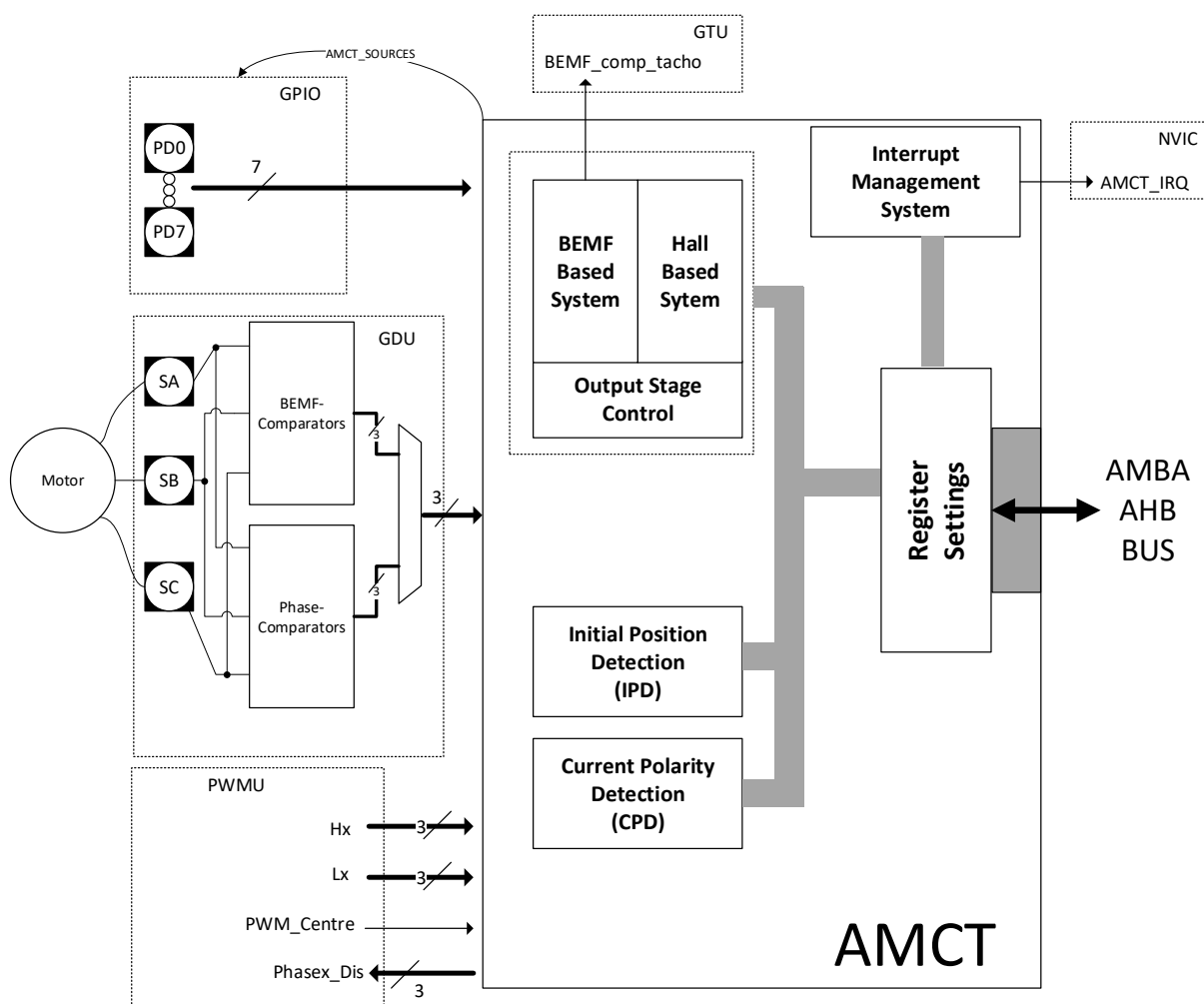


Figure 1-1: Block Diagram of the AMCT Module

Automotive SoC Motor Driver

In addition, a bemf tacho signal, bemf_comp_tacho, is generated from AMCT module to general timer unit (GTU) in order to measure the frequency of motor when windmilling.

The hall-based system contains debounce filters to remove noise from hall sensor output, a dedicated timer to measure combination of hall sensors output frequency as well as a dedicated phase shift (phase delay or advance) generator.

Phase disable signals, Phasex_Dis, generated by AMCT module can be either generated from hall-based system or bemf based system, and this stage is controlled in the “output stage control” subsystem. Lastly, interrupt sources can be generated from AMCT module for signals that are critical, and these interrupt sources are managed in the “interrupt management system”. In the following sections each sub-system is described extensively and the input and output from each sub-module is defined.

Automotive SoC Motor Driver

1.4 Hall Based System

For motor control applications which requires hall sensors, the hall based system can be utilized to filter the noise associated with the PWM switching in the hall sensors, calculate hall frequency, and achieve phase advance or delay operation. The block diagram of the hall based system is shown in Figure 1-2. The block becomes functionally active once *HAEN* is set to '1'. When *HAEN* is set to '0' all the outputs from this subsystem are reset to their default condition.

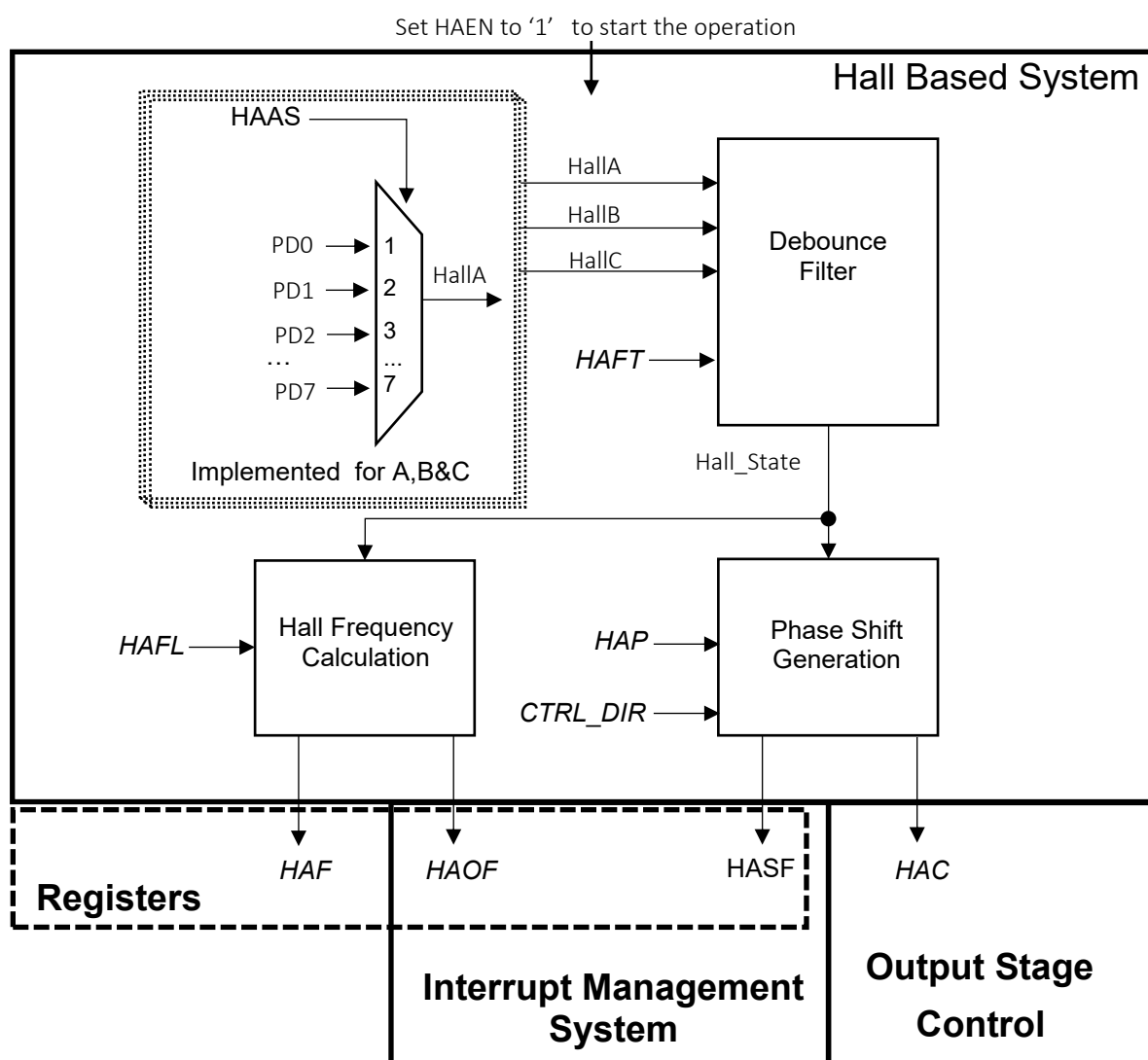


Figure 1-2: Hall Based System Block Diagram

Automotive SoC Motor Driver

1.4.1 Hall Sensor GPIO pin connection

The first step is to choose the GPIO pins to which the hall sensors connected. In total the hall-based system expects three hall sensors, one for each motor phase. The hall selection register settings HAAS, HABS and HACS, defines the GPIO pins for each respective HallA, HallB and HallC. It should be noted if a system uses only one hall sensor or two hall sensors, the hall based system can still be used to filter the noise in the hall sensor output and measure its frequency if required.

Note: The GPIO pins which are used for hall sensors needs to be configured as input. To learn more about configuring the GPIO pins as input read GPIO documentation.

1.4.2 Debounce Filter

The debounce filter is mainly used to avoid problems associated with PWM switching noise. The debounce filter operation starts once HAEN is set to 1. If the combination of the three hall inputs (HallA, HallB and HallC) doesn't change for the duration hall filter time, t_{HFT} , the debounce filter will update the output Hall_State representing a 3 bit number. The Hall_State is formed from the combination of HallA, HallB, and HallC states, where HallC is the most significant bit, HallA is least significant bit and HallB is the middle bit. If any of the three hall inputs changes within the t_{HFT} , debounce filter will not update the Hall_State, hall filter time resets and current Hall_State value will be maintained. This behavior is shown in Figure 1-3.

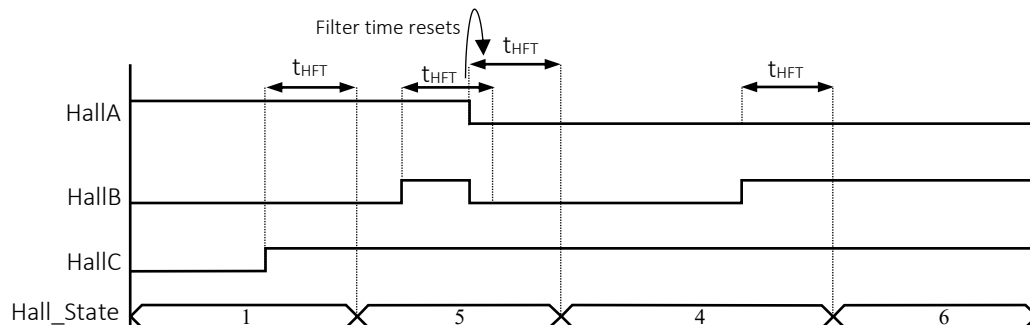


Figure 1-3: Hall Debounce Filter Functional Behavior

The time t_{HFT} can be set by HFT variable in AMCT Hall Filter Time register and can be calculated according to the equation below:

$$t_{HFT} = \frac{HFT}{f_{AMCT}} \text{ s}$$

Where f_{AMCT} represent the clock frequency in AMCT module and by default set to 40Mhz.

Automotive SoC Motor Driver

For example if it is desired to achieve the hall filter time, t_{HFT} , of 300ns, HFT can be calculated as below assuming f_{AMCT} is 40MHz.

$$HFT = t_{HFT} \times f_{AMCT} = 40MHz \times 300ns = 12$$

Table 1-1 demonstrates the relationship between Hall_State and Hall inputs.

Table 1-1: Hall Inputs and Hall_State Relationship after debounce filter time

HallC	HallB	HallA	Hall_State [2:0] (if inputs do not change for t_{HFT} and HAEN=1)
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

Automotive SoC Motor Driver

1.4.3 Hall Frequency Calculation

A timer is used to track the rate at which the Hall_State changes when HAEN bit is set to 1. This rate of change can be calculated according to the following equation:

$$\text{Hall_State (rate of change)} = \frac{f_{AMCT}}{HAF - 1} \text{ Hz}$$

For example, When f_{AMCT} is 40MHz and HAF value of 1000000 is read, the Hall_State value is changing at the following rate:

$$\text{Hall_State(rate of change)} = \frac{40\text{MHz}}{100000} = 400 \text{ Hz}$$

Figure 1-4 shows the functional behavior of hall frequency calculation. HAF value is reset to 1 each time the hall block is disabled (HAEN=0). When HAEN is set to 1, HAF is for the first time updated the second time Hall_State changes. The HAF is then updated following each change in Hall_State. It should be noted, the minimum rate of change that can be captured is limited by a 32-bit number HAFL. For example, assuming HAFL is set to its maximum value, the minimum frequency that can be measured, with f_{AMCT} at 40MHz, can be calculated as below:

$$\text{Hall_State(minimum rate of change)} = \frac{40\text{MHz}}{2^{32} - 1} \approx 9 \text{ mHz}$$

If HAF reaches to the value defined by HAFL (t_{HAFL}), HAF will be reset to 1 and the overflow flag bit HAOF will be set in the AMCT_STATUS register indicating the combination of hall sensors input states haven't changed for a period longer than set by HAFL parameter. HAOF bit can be cleared by successful write of 1 to HAOF bit. The source of HAOF flag will be cleared once the Hall_State value changes. When HAOF is flagged, HAF maintains the reset value of 1 and will only get updated to a new value once two Hall_State changes occurs. The HAF is then updated following each change in Hall_State. The HAOF is also sent to interrupt management system to generate interrupt upon this overflow condition if the interrupt generation for this bit is enabled. More information about interrupt generation available in the interrupt management system section.

Automotive SoC Motor Driver

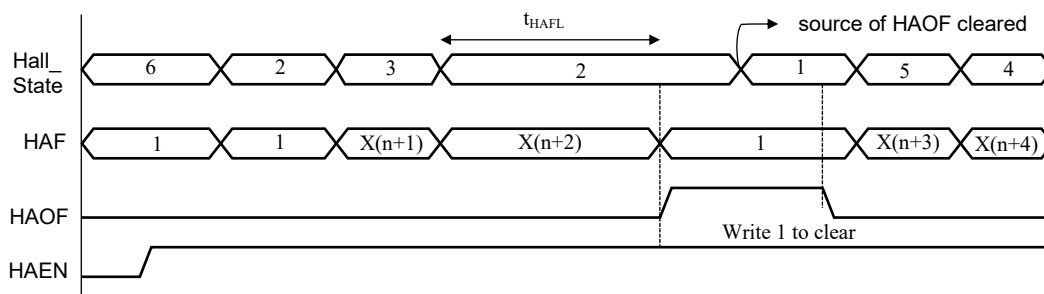


Figure 1-4: Hall Frequency Calculation Functional Behavior

1.4.4 Phase Shift Generation

The HACS output shown in Figure 1-2 can be updated with Hall_State, delayed version of Hall_State, or advance version of Hall_State. This allows achieving phase advance or delay operation for motor control. The amount of phase shift and the shift mode depends on the signed 32-bit variable HAPS. When HAPS is a positive number, the shift mode is set to phase delay and when HAPS is a negative number, the shift mode is set to phase advance. When HAPS is set to zero, no phase shift is generated and HACS value is updated immediately with Hall_State. To start phase shift generation HAEN should be set to 1. If HAEN is set to 0, no phase shift will be generated regardless of HAPS setting and HACS is reset to 0.

1.4.4.1 Phase Advance Operation

When HAPS is a negative number, the shift mode is set to phase advance. According to the current state of the Hall_State, it is possible to predict the next value of Hall_State. The relationship between Hall_State and the expected Hall_State value is provided in Table 1-2. It should be noted the sequence for expected Hall_State value depends on the motor rotation direction, CTRL_DIR, and hence the table provides the value for both motor rotation direction.

Automotive SoC Motor Driver

Table 1-2: Hall State and Expected Hall State Relationship

HallC	HallB	HallA	Hall_State	Expected Hall_State	CTRL_DIR
0	0	0	0	0	1
0	0	1	1	3	1
0	1	0	2	6	1
0	1	1	3	2	1
1	0	0	4	5	1
1	0	1	5	1	1
1	1	0	6	4	1
1	1	1	7	7	1
0	0	0	0	0	0
0	0	1	1	5	0
0	1	0	2	3	0
0	1	1	3	1	0
1	0	0	4	6	0
1	0	1	5	4	0
1	1	0	6	2	0
1	1	1	7	7	0

Figure 1-5 shows the functional behavior of the phase advance operation in both *CTRL_DIR* setting. When *HAEN* is set to 0, *HACS* is reset to 0. When *HAEN* is set to 1, *HACS* will be updated immediately with the *Hall_State*. The phase advance operation then will be functionally active once at least one change in *Hall_State* is observed which is marked as point A. The amount of phase advance can be adjusted by modifying the variable *HAPS*. Each time *HAPS* is modified, the modification will be effective on the next *Hall_State* change. For example, when *HAPS* is changed while *Hall_State* value is 5, the change will be functionally effective once *Hall_State* value is changed to 1 for the condition *CTRL_DIR*=1.

Automotive SoC Motor Driver

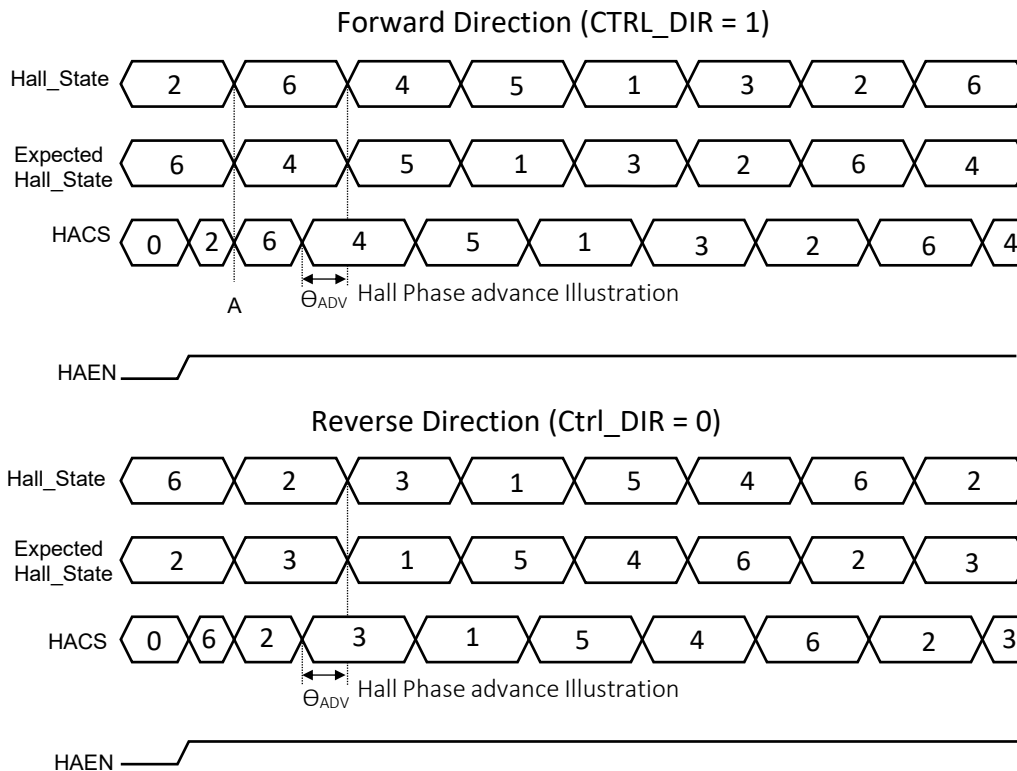


Figure 1-5: Phase Advance Functional Behavior

As shown in Figure 1-5, *HACS* is updated with the expected *Hall_State* value by Θ_{ADV} degree in advance. Θ_{ADV} value depends on the variable *HAPS* and can be calculated as below:

$$\Theta_{ADV} = \frac{(HAF + HAPS)}{HAF} \times 60 \text{ degree}$$

Where *HAF* represents the hall sensors rate of change in the hall result register.

For example, if it is desired to set phase advance Θ_{ADV} to 20 degree and assuming *HAF* is read as 5000, then *HAPS* can be calculated as below:

From above equation:

$$\begin{aligned} HAPS &= \frac{HAF}{60} \times \Theta_{ADV} - HAF \\ &= \frac{5000}{60} \times 20 - 5000 \approx -3333 \end{aligned}$$

Therefore in software *HAPS* should be set to -3333.

Automotive SoC Motor Driver

It should be noted the maximum allowed phase advance Θ_{ADV} should be limited to 60 degree when implementing the calculation.

1.4.4.2 Phase Delay Operation

When HAPS is a positive number, the shift mode is set to phase delay. In phase delay operation, the HACS value is updated by Hall_State after Θ_{DLY} degree. The functional behaviour for phase delay operation is shown in Figure 1-6. When HAEN is set to 1, HACS will be updated immediately with the Hall_State. The phase delay operation then will be functionally active once at least one change in Hall_State is observed which is marked as point A. The amount of phase delay can be adjusted by modifying the variable HAPS. Each time HAPS is modified, the modification will be effective on the next Hall_State change. For example, when HAPS is changed while Hall_State value is 5, the change will be functionally effective once Hall_State value is changed to 1 for the condition CTRL_DIR=1.

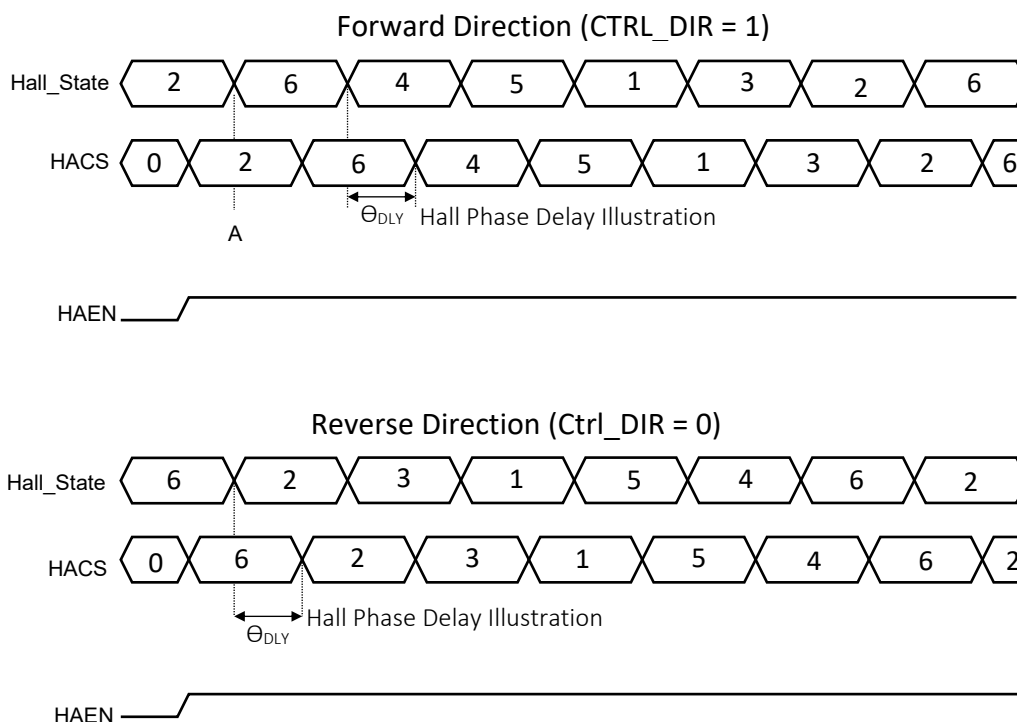


Figure 1-6: Phase Delay Functional Behavior

Automotive SoC Motor Driver

The phase delay amount Θ_{DLY} depends on the HAPS and can be calculated as below:

$$\Theta_{DLY} = \frac{HAPS}{HAF} \times 60 \text{ degree}$$

Where HAF can be read from AMCT Hall Frequency register.

For example, if it is desired to set phase delay, Θ_{DLY} , to 20 degree and assuming HAF is read as 5000, then HAPS can be calculated as below:

$$\begin{aligned} HAPS &= \frac{HAF}{60} \times \Theta_{DLY} \\ &= \frac{5000}{60} \times 20 \approx 1666 \end{aligned}$$

It should be noted the maximum allowed phase delay Θ_{DLY} should be limited to 60 degree when implementing the calculation. The phase delay will be automatically clamped to 60 degree should any phase delay greater than 60 degree be requested.

1.4.5 Hall State Fault (HASF)

Whenever *Hall_State* signal represent the illegal state, Hall State Fault (HASF) flag will be set to 1 in AMCT_STATUS register. States 0 and 7 are considered illegal for the *Hall_State* as the combination of three hall inputs can never be all 0 or all 1 under normal circumstances. The fault can be cleared by writing 1 to the corresponding bit in the AMCT_STATUS register. The source of the fault (*Hall_State* = 0 or *Hall_State* = 7) needs to be removed before clearing the hall state fault flag or otherwise it will be set again. The phase shift generation and hall frequency calculation will proceed as before regardless of this fault condition.

Automotive SoC Motor Driver

1.5 BEMF Based System

The top-level diagram of the BEMF based system is shown in Figure 1-7. The BEMF based system contains a dedicated timer which is used to generate a parameter, MDA, related to motor driving motor angle which can be used for motor control algorithm. The angle generator sub-module also provides a commutation section parameter, MDA_CS, like hall-based system, which can be utilized to energize the motor phases in a correct sequence. Refer to output stage control section for more information about the relationship between the MDA_CS and the motor phases to be energized. In addition, this sub-module has a dedicated filtering stage to remove the degaussing effect, from motor winding de-energization during, and the noise from bemf comparators output. A tacho signal, BEMF_comp_tacho is also generated from the combination of the three bemf comparators which can be utilized to estimate the motor speed when windmilling. A dedicated timer is also allocated to calculate the bemf zero crossing error, BEMFAE, with respect to the angle generated by the parameter MDA.

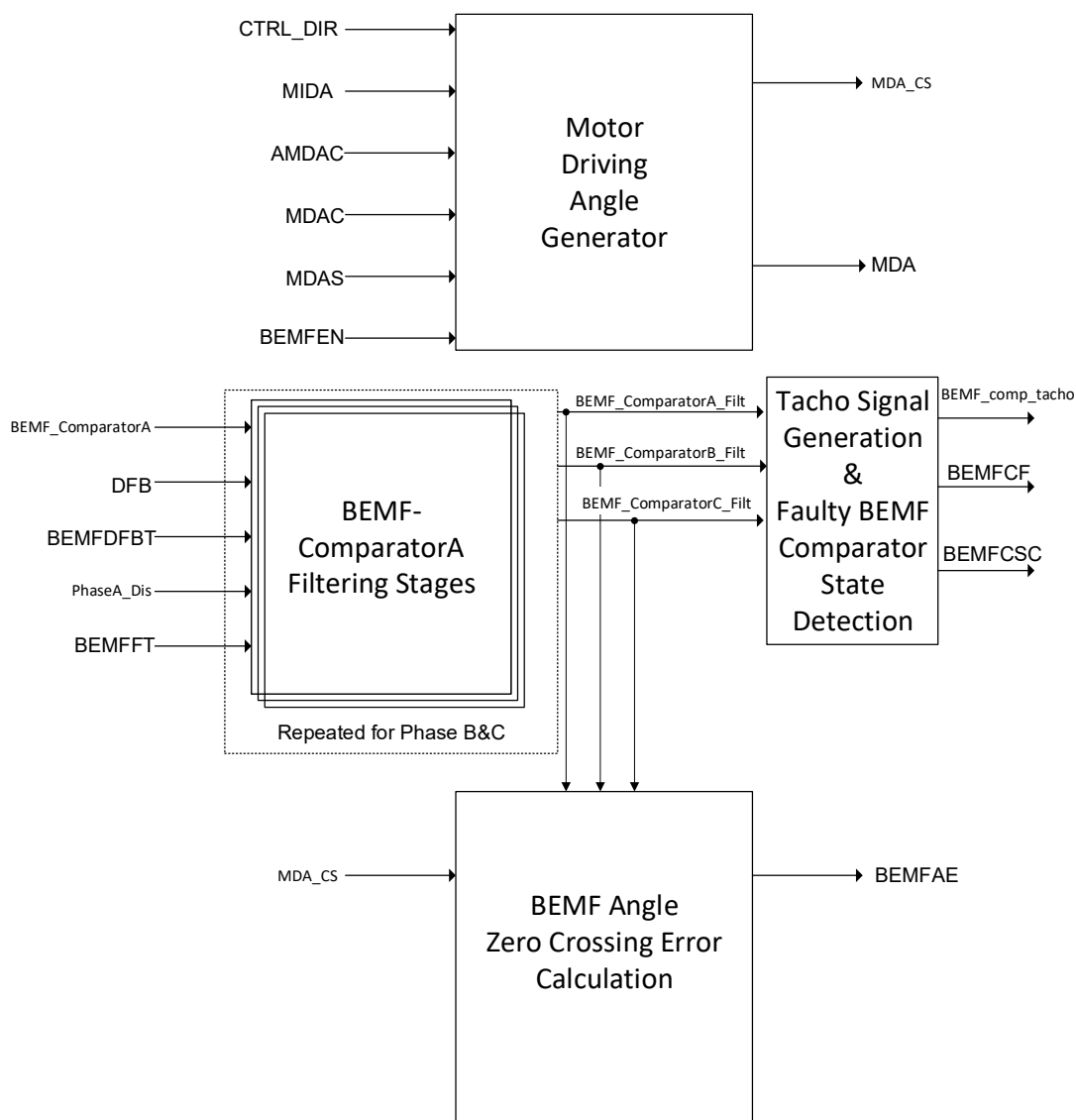


Figure 1-7: BEMF Based System Top Level Diagram

Automotive SoC Motor Driver

In summary the BEMF based system facilitates two important functionalities, window-based sensorless BEMF control and windmilling detection speed. This subsystem works for control algorithm that relies on 60-degree bemf window opening. Also, for applications which are based on sensorless field-oriented control (FOC) the angle generator can be used for the electrical angle used in FOC algorithm. This can save some memory space and optimize algorithm speed slightly. In the following sections each of the sub-system is explained in greater details.

1.5.1 Motor Driving Angle Generator

In the motor control algorithm, a parameter is required to track the angle which is used to determine the correct commutation for motor phases. A dedicated timer developed in AMCTU can be used to generate the parameter, MDA, relating to motor angle. There is flexibility to define the initial value of the parameter MDA, the rate at which it increases, which is directly related to the motor frequency, and apply correction to MDA parameter as the control algorithm may find it necessary during feedback control. The relationship between the parameter MDA and angle in degree can be obtained from the below equation:

$$Angle = \frac{MDA}{2^{32} - 1} \times 360 \text{ (degree)}$$

The rate at which MDA changes is related to the electrical motor frequency and is defined by the parameter MDAS according to below equation:

$$MDAS = \frac{f_{Motor-Electrical}}{f_{AMCT}} \times 2^{32}$$

where:

$f_{Motor-Electrical}$ = motor electrical frequency in Hz decided by control algorithm

f_{amct} = AMCT module clock frequency which by default is set to 40Mhz

for example, if the motor electrical frequency is 40 Hz, then MDAS should be programmed to be:

$$MDAS = \frac{40}{40 \times 10^6} \times 2^{32} = 4295$$

The rate of change of MDA parameter can be either positive or negative depending on the motor control direction defined by CTRL_DIR. When CTRL_DIR is set to '0' the MDA parameter is decremented at a rate defined by MDAS. When CTRL_DIR is set to '1', the MDA parameter is incremented at a rate defined by MDAS.

Automotive SoC Motor Driver

In addition, sometimes it is necessary to apply correction to motor angle as the controller receives feedback information. If the driving angle MDA, is lagging the actual motor position, then a positive correction factor MDAC needs to be applied to the parameter MDA. and when the MDA is leading the actual motor positive a negative MDAC needs to be applied to the parameter MDA. The correction factor is applied upon detection of rising edge in the programmable parameter AMDAC.

Figure 1-8 and Figure 1-9 Show the waveforms which describes the functional behaviour of the angle generation block depending on the motor direction.

Figure 1-8 shows the function behaviour of the motor angle generator when CTRL_DIR is set to '0'. The angle generator starts operating when BEMFEN is set to '1'. The initial value of the motor driving angle, MDA, is defined by the parameter MIDA. The MDA is then decremented at a rate defined by the parameter MDAS. At time A, upon detection of rising edge of the programmable parameter AMDAC, when MDAC is negative and CTRL_DIR is set to '0', the absolute value of MDAC is added to the MDA. When the MDA parameter reaches to 0, it resets to the max value of 2^{32} and the sequence repeats. At time B, when MDAC is positive and CTRL_DIR is set to '0', upon detection of another rising edge AMDAC, the absolute value of MDAC is subtracted from MDA. When BEMFEN is set to '0', the MDA resets to 0 and the angle generation is disabled.

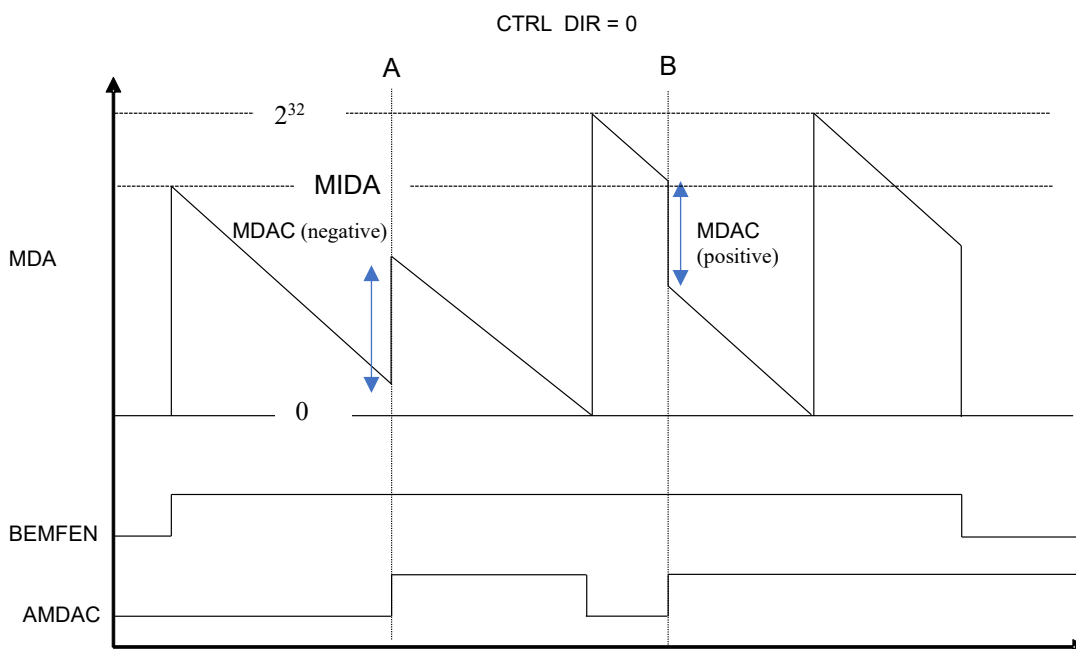


Figure 1-8: Motor Driving Angle Generation Functional Behavior (CTRL_DIR = 0)

Automotive SoC Motor Driver

Figure 1-9 shows the function behavior of the motor angle generator when CTRL_DIR is set to '1'. The angle generator starts operating when BEMFEN is set to '1'. The initial value of the motor driving angle, MDA, is defined by the parameter MIDA. The MDA is then incremented at a rate defined by the parameter MDAS. At time 'A', upon detection of the rising edge of the programmable parameter AMDAC, when MDAC is negative and CTRL_DIR is set to '0', the absolute value of MDAC is subtracted from the MDA. When the MDA parameter reaches to the max value ' 2^{32} ', it resets to 0 and the sequence repeats. At time B, when MDAC is positive and CTRL_DIR is set to '0', upon detection of another rising edge AMDAC, the absolute value of MDAC is added to MDA. When BEMFEN is set to '0', the MDA resets to 0 and the angle generation is disabled.

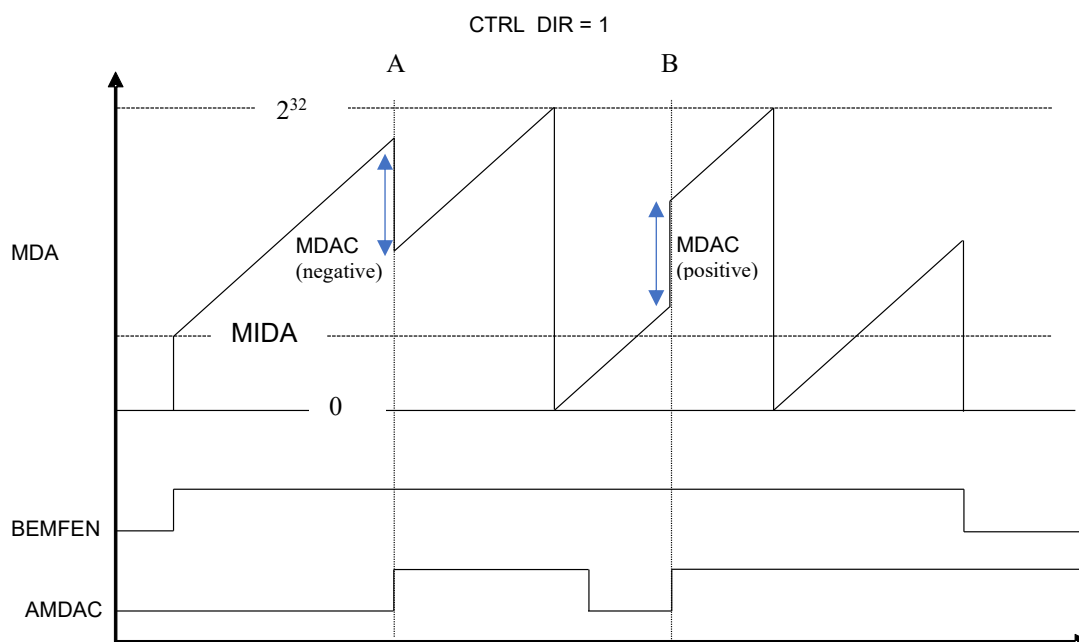


Figure 1-9: Motor Driving Angle Generation Functional Behavior (CTRL_DIR = 1)

Driving angle generation sub-module also provides commutation section, MDA_CS, which can be used to commute the motor phases in a correct sequence. The relationship between commutation section, MDA_CS, and the motor phases to be energized is discussed in the output stage control section. The relationship between commutation section, MDA_CS and the motor driving angle, MDA, is provided in Table 1-3.

Automotive SoC Motor Driver

Table 1-3: Commutation Section and Motor Driving Angle Relationship

MDA	Angle (Degree)	MDA_CS
$0xEAAAAAAB \leq MDA < 0x15555555$	$330 \leq \text{Angle} < 30$	3
$0x15555555 \leq MDA < 0x40000000$	$30 \leq \text{Angle} < 90$	2
$0x40000000 \leq MDA < 0x6AAAAAAB$	$90 \leq \text{Angle} < 150$	6
$0x6AAAAAAB \leq MDA < 0x95555555$	$150 \leq \text{Angle} < 210$	4
$0x6AAAAAAB \leq MDA < 0xC0000000$	$210 \leq \text{Angle} < 270$	5
$0x6AAAAAAB \leq MDA < 0xEAAAAAAB$	$270 \leq \text{Angle} < 330$	1

Automotive SoC Motor Driver

1.5.2 BEMF Comparator Filtering Stages:

The block diagram of BEMF filtering stage is shown in Figure 1-10. When a motor phase is disabled, the existing current in the motor winding phase can result in the degaussing effect in the comparator output due to the time it takes for the current to discharge. The degaussing filterer stage will remove the degaussing effect. The degaussing filter **only** works when the motor angle is controlled by the motor angle generator described in the last section and the motor phases are energized according to criteria explained in the output stage control section. The degauss filter stage starts by setting BEMFEN and Phasex-Dis to '1'. It should be noted that Phasex-Dis should be automatically controlled by the output stage control section and the user shouldn't worry about this signal. It is also possible to manually control Phasex-Dis signal, however, degauss filtering stage will not work in this scenario. Therefore for applications which requires manual selection of Phasex-Dis signal, degauss filtering stage should be bypassed by setting DFB to '1'. Following degauss filtering stage, a debounce filter is used to remove the noise associated with bridge FETs switching effect close to BEMF zero crossing point. The following sections provide greater details about each filtering stage. In summary, for motor control algorithm which is based on 60 degree trapezoidal control degaussing and debouncing filter can be used together. For obtaining windmilling algorithm debounce filter can be used alone and the degauss filter should be bypassed. For field oriented control algorithm, the information about bemf comparator outputs are usually not required and these filtering stages are not required to be used.

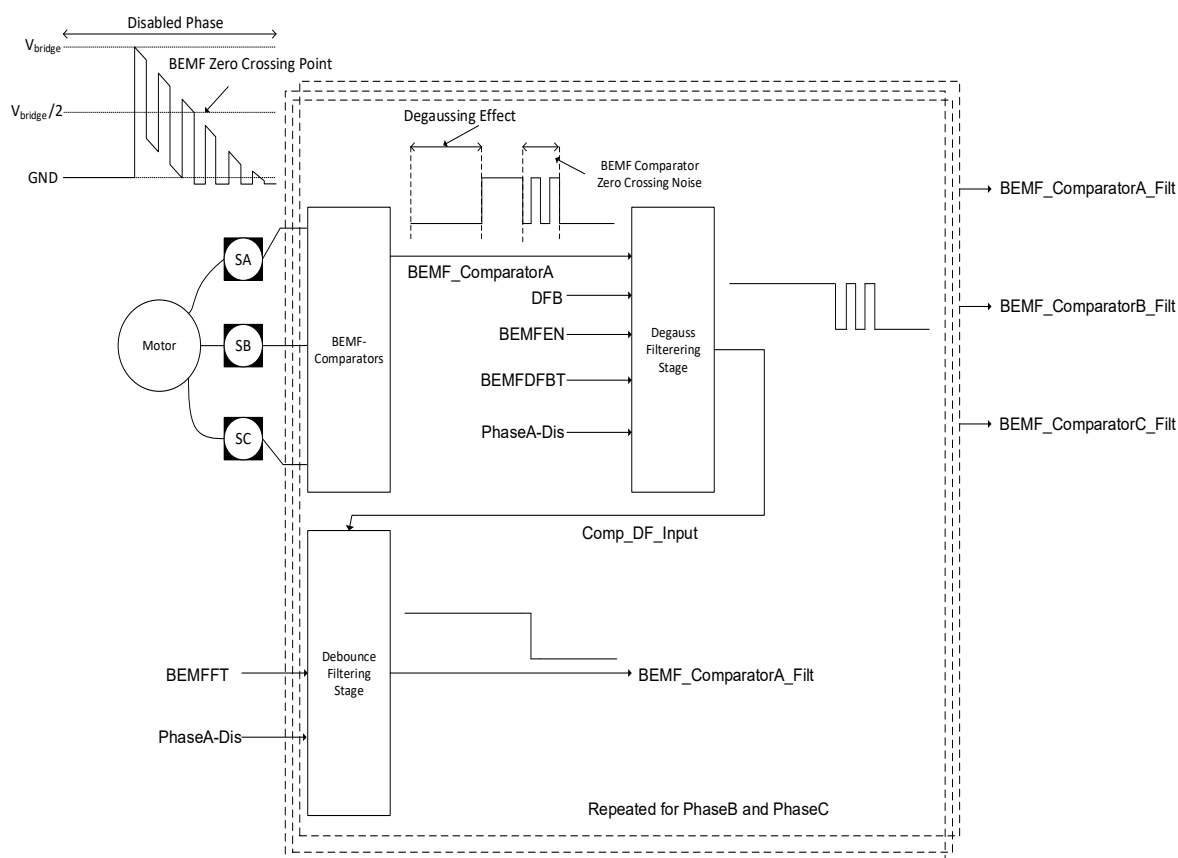


Figure 1-10: BEMF Comparator Filtering Stage Block Diagram

Automotive SoC Motor Driver

1.5.2.1 BEMF Degauss Filter:

When a phase is just disabled, it takes some time for the current in the winding to decay to '0'. This winding de-energization results in the degaussing effect in the bemf comparator output. The degauss filtering stage removes the undesired degaussing effect in the bemf comparator output. The degauss filtering stage starts once the BEMFEN and Phasex_Dis are set to '1'. The Phasex_Dis signal is automatically controlled by the output stage control section and the user does not need to worry about this parameter. The settings need to be adjusted in the output stage control section correctly which is discussed in the following sections. It is also possible to manually control Phasex_Dis signal, however, degauss filtering stage will not work in this scenario. Therefore for applications which requires manual selection of Phasex_Dis signal, degauss filtering stage should be bypassed by setting DFB to '1'. The parameter BEMFDFBT defines the bemf degauss filter blank time and it removes the noise associated with the turn off of the bridge FETs for a corresponding phase. This parameter generally should be set to a value 10% higher than the time required to fully turn off the bridge FETs. The BEMF degauss filter blank time can be calculated according to the equation below:

$$\text{Degauss Filter Blank Time} = \frac{\text{BEMFDFBT}}{f_{AMCT}} \text{ s}$$

Where f_{AMCT} represent the clock frequency in the AMCTU which by default is set to 40Mhz.

For example if it takes 400ns for the bridge FETs to turn off, then the blank time should be set to 10% higher than 400ns and BEMFDFBT can be calculated as below:

$$\text{BEMFDFBT} = 440 \times 10^{-9} \times 40 \times 10^6 \cong 18$$

When BEMFEN is set to '0', the degauss filtering stage is disabled and the output of this filter is set to '0'.

1.5.2.2 BEMF Debounce Filter:

Figure 1-11 shows the functional behaviour of the debounce filter. The debounce filter starts operating once BEMFEN and Phasex_Dis are set to '1'. If the input, Comp_DF_Input doesn't change for the duration t_{BEMFT} , the debounce filter output, BEMF_Comparator_Filt is updated with the Comp_DF_input value. However, if the Comp_DF_input changes before filter time t_{BEMFT} , expires, the debounce filter output, BEMF_Comparator_Filt, maintains its current value and filter time t_{BEMFT} resets. When Phasex_Dis signal is set to '0' as shown at time 'A', BEMF_Comparator_Filt maintains its previous value regardless of any change in the Comp_DF_Input. When BEMFEN is set to '0', the debounce filter is disabled and the debounce filter output, BEMF_Comparator_Filt is reset to 0. Therefore debounce filter is fully functional once both BEMFEN and Phasex_Dis signals are set to '1'. The Phasex_Dis signal generation can be either automatically generated or controlled manually. This operation is controlled by the output stage control sub-module and is described in the following sections. The filter time t_{BEMFT} is calculated by the parameter BEMFFT and can be calculated as below:

$$t_{BEMFT} = \frac{\text{BEMFFT}}{f_{AMCT}} \text{ (s)}$$

Where f_{AMCT} is the clock frequency of AMCT block and by default it is set to 40Mhz.

Automotive SoC Motor Driver

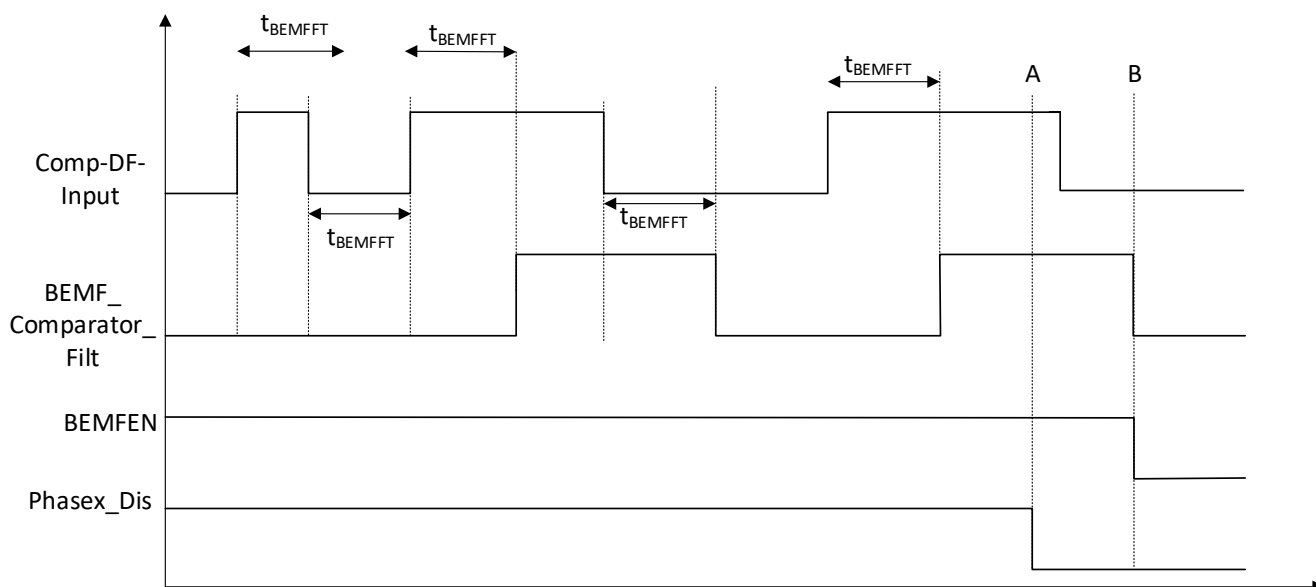


Figure 1-11: Debounce Filter Functional Behavior

1.5.3 BEMF Angle Zero-Crossing Error Calculation:

There is a dedicated timer used that measures the BEMF-zero crossing angle error, BEMFAE, with respect to the motor driving angle, MDA, used in the control algorithm. The angle error calculation starts once the BEMFEN is set to '1'. When the BEMFEN is set to '0', BEMFAE is reset to 0. When a motor phase is disabled by setting Phasex_Dis signal to '1', the bemf information can be obtained from a corresponding phase. Since the Phasex_Dis signals are automatically generated as will be described in the output stage control section, the BEMF angle error can be obtained with respect to driving angle, MDA, by observing how soon or late the BEMF zero-crossing occurs. Therefore a correction factor can be added or subtracted to MDA, from the reported bemf angle error. BEMFAE is updated each time a new commutation section, MDA_CS, as described in Table 1-1, occurs and therefore in total 6 BEMF zero crossing information is provided per electrical cycle. The angle error between driving angle and bemf angle can be calculated according to the below equation:

$$\text{Angle Error} = \frac{\text{BEMFAE} \times \text{MDAS}}{2^{32} - 1} \times 360 \text{ degree}$$

It should be noted BEMFAE is a signed number. A negative number indicates the bemf angle is lagging the driving motor angle, MDA. A positive number indicates the motor bemf angle is leading the driving motor angle. With the knowledge of angle error, the user can apply correct factor, MDAC, to MDA, to achieve phase delay, phase advance or in-phase operation as needed.

Important notes:

- 1- The BEMFAE is correctly reported only when angle generator, MDA, described in the previous sections, is used for control algorithm.
- 2- Motor phases should be energized according to commutation section, MDA_CS and Phasex_Dis signals should be automatically generated. Output Stage control section provides details on how Phasex_Dis signals are automatically generated and which motor phases to energize depending on MDA_CS parameter.

Automotive SoC Motor Driver

1.5.4 Tacho Signal Generation:

It is possible that when all three motor phases are disabled, PhaseA_Dis, PhaseB_Dis and PhaseC_Dis set to '1', the motor still rotating due to environmental conditions known as windmilling. Also in applications such as cooling fan, when the motor phases are disabled, the fan slowly comes to stop depending on the rotor inertia and if the controller is enabled immediately after, the fan may still be rotating. In this scenario the Tacho signal generated and also the state bemf_comparators_Filt can help identifying the motor position as well as its rotating speed before starting the control algorithm. Let's consider a condition where the motor is windmilling while all three motor phases are disabled. In this scenario, the voltage across the SA,SB and SC nodes as well as the state of the BEMF comparators can be described by the waveforms in Figure 1-12. The state of the combined three BEMF outputs after filtering stages can be obtained by reading the parameter FBEMFCO in BEMF_COMPARATOR_FILTERED register. FBEMFCO is three bit number where bit 2 defines the state of BEMF_ComparatorC_filt, bit 1 defines the state of BEMF_ComparatorB_filt and bit 0 defines the state of BEMF_ComparatorA_filt. Since the voltages V_{sa} , V_{sb} and V_{sc} represent the motor BEMF voltages offset by motor neutral point voltage $V_{neutral}$, it is possible to estimate the motor position when windmilling by reading the parameter FBEMFCO. For example when FBEMFCO is set to '101_b', it means that the motor is between 0 to 60 degree. A tacho signal, BEMF_comp_tacho, is also generated which toggles each time the state of any of three bemf comparators is changed. This tacho signal is connected to general timer unit (GTU) and GTU can be used to measure the frequency of tacho signal and therefore estimate the motor windmilling frequency. Refer to GTU documentation for further information. The bemf_comp_tacho and also all the three comparator outputs both after and before filtering stage are available as an output source signal in the GPIO module and can be output from the GPIO ports. Refer to GPIO documentation to learn about proper settings for outputting these signals from GPIO pins if desired.

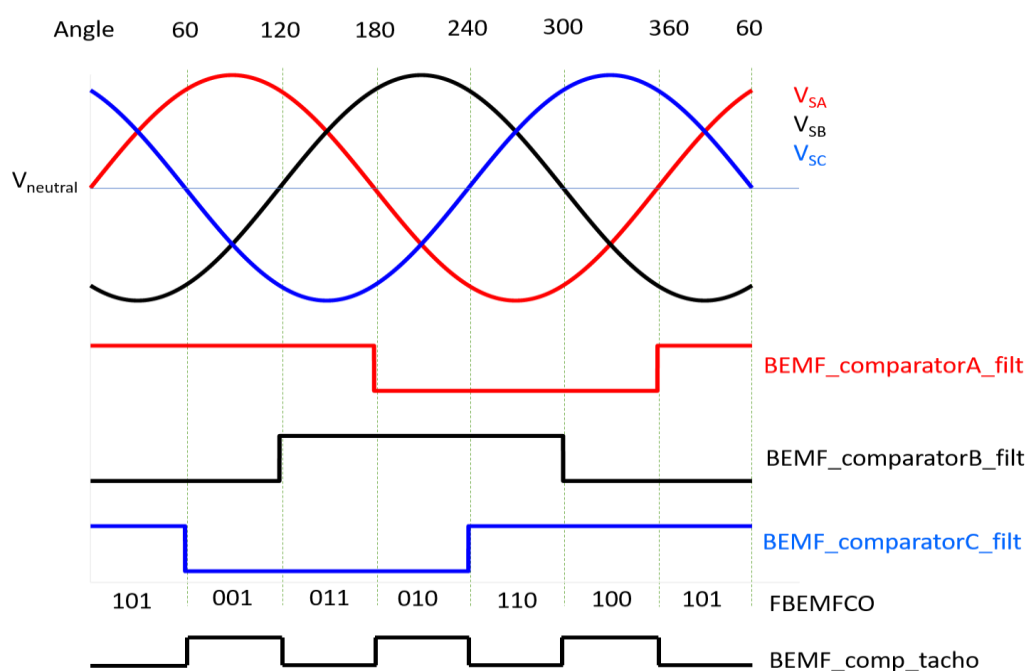


Figure 1-12: Phase Voltages and BEMF Comparator Outputs During Windmilling

Automotive SoC Motor Driver

1.5.5 Fault BEMF Comparator Output (BEMFCF)

When the motor is windmilling, there is no condition in which the combination of all three bemf comparator outputs reported by parameter FBEMFCO, are '0' or '1'. Therefore, if all the comparator outputs after filtering stages are set to '0' or '1' (FBEMFCO = '000' or '111') BEMFCF is flagged in the AMCT_STATUS register. This parameter can be cleared by writing '1' to the BEMFCF bit in the AMCT_STATUS register or by setting BEMFEN bit to '0'. This bit is also transferred to interrupt management system to generate an interrupt if desired. Refer to interrupt management system section for further information.

1.5.6 BEMF Comparators Output Stage Change (BEMFCSC)

When the state of any of the three BEMF comparator outputs are changed after filtering stages, the bit BEMFCSC is flagged in the AMCT_STATUS register. For example, when FBEMFCO is changed from '101_b' to '001_b' BEMFCSC is flagged. This bit can be cleared by successful write of '1' to the BEMFCSC bit in the AMCT_STATUS register or by setting BEMFEN bit to '0'. This bit is also transferred to interrupt management system to generate an interrupt each time the state of any of the three BEMF comparator outputs are changed. Refer to interrupt management system for more information.

Automotive SoC Motor Driver

1.6 Output Stage Control

Output stage control will generate the phase disable signal outputs, PhaseA_Dis, PhaseB_Dis and PhaseC_Dis which disables a corresponding motor phase according to the settings in the pulse width modulation unit (PWMU). Refer to PWMU documentation for further information. These Phase disable signals are also sent as a feedback to BEMF based system to calculate BEMF angle zero crossing error and is also used for BEMF comparator output filtering stages. Figure 1-13 shows the block diagram of the output stage control. The phase disable signal generation starts by setting the MASTEREN bit to '1'. It is possible to generate the phase disable signals manually by programming a register or let the dedicated hardware in BEMF based system or Hall based system generate the phase disable signals. When phase disable mode select, PDMS, is set to '0', phase disable signals are generated automatically by signal AP_DIS which is generated from either hall-based system or BEMF based system. When phase disable mode select, PDMS, is set to '1', phase disable signals are generated by programming the parameter MP_DIS in the MANUAL_PHASE_DISABLE register. The parameter AP_DIS is generated according to the mux setting AMCTCS. The commutation section generated by AMCT module, AMCTCS, is generated by the hall-based system with the signal output HAC when commutation section control mode, CSCM is set to '0'. When CSCM is set to '1', AMCTCS is generated by the signal MDA_CS. In order to use angle error calculation and degauss filtering stage in the BEMF based system, it is very important that the parameters CSCM is set to '1' and PDMS is set to '0', in order to ensure the phase disable signals are automatically generated by the BEMF based system. To use tacho signal generation feature in BEMF-based system it is important to generate all phase disable signals manually by setting PDMS parameter to '1' and programming MP_DIS parameter to '111'. The signals PDRE_A, PDRE_B and PDRE_C are latched to '1' up-on detection of rising edge on corresponding PhaseA_Dis, PhaseB_Dis, and PhaseC_Dis signals. The latched signals can be read from AMCT_STATUS register. The latched signal can only be reset by successful write of '1' to corresponding bit in AMCT_STATUS register or setting MASTEREN bit to 0. The signals PDRE_A, PDRE_B and PDRE_C are also transferred to interrupt management system to generate an interrupt if desired.

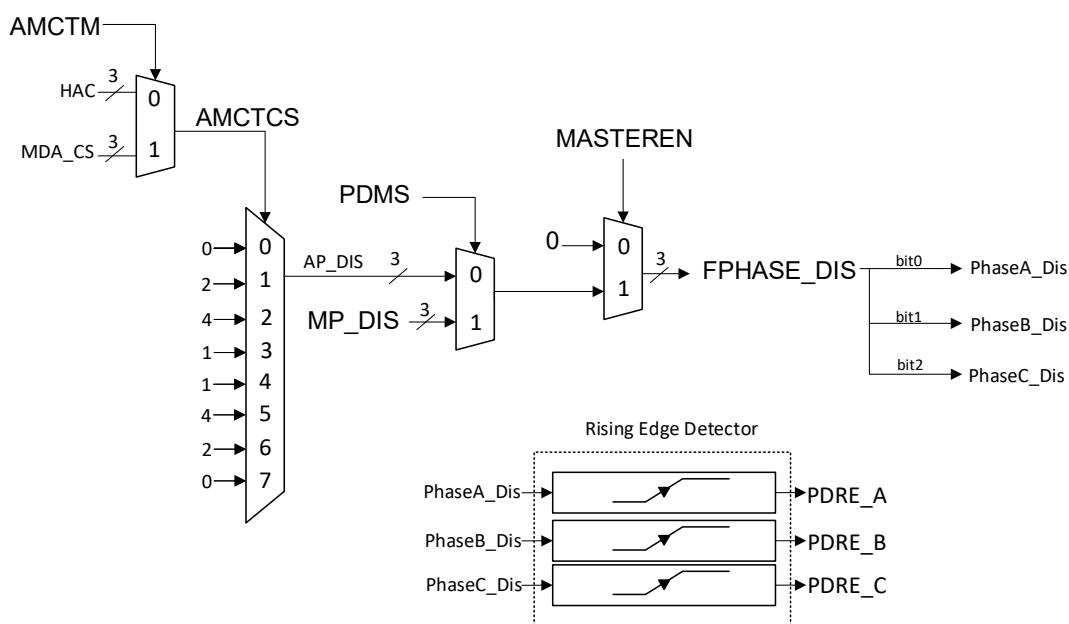


Figure 1-13: Output Stage Control Block Diagram

Automotive SoC Motor Driver

Figure 1-14 suggests the commutation point for motor phases according to the AMCTCS generated automatically from either BEMF based system or Hall-based system to achieve BLDC trapezoidal control.

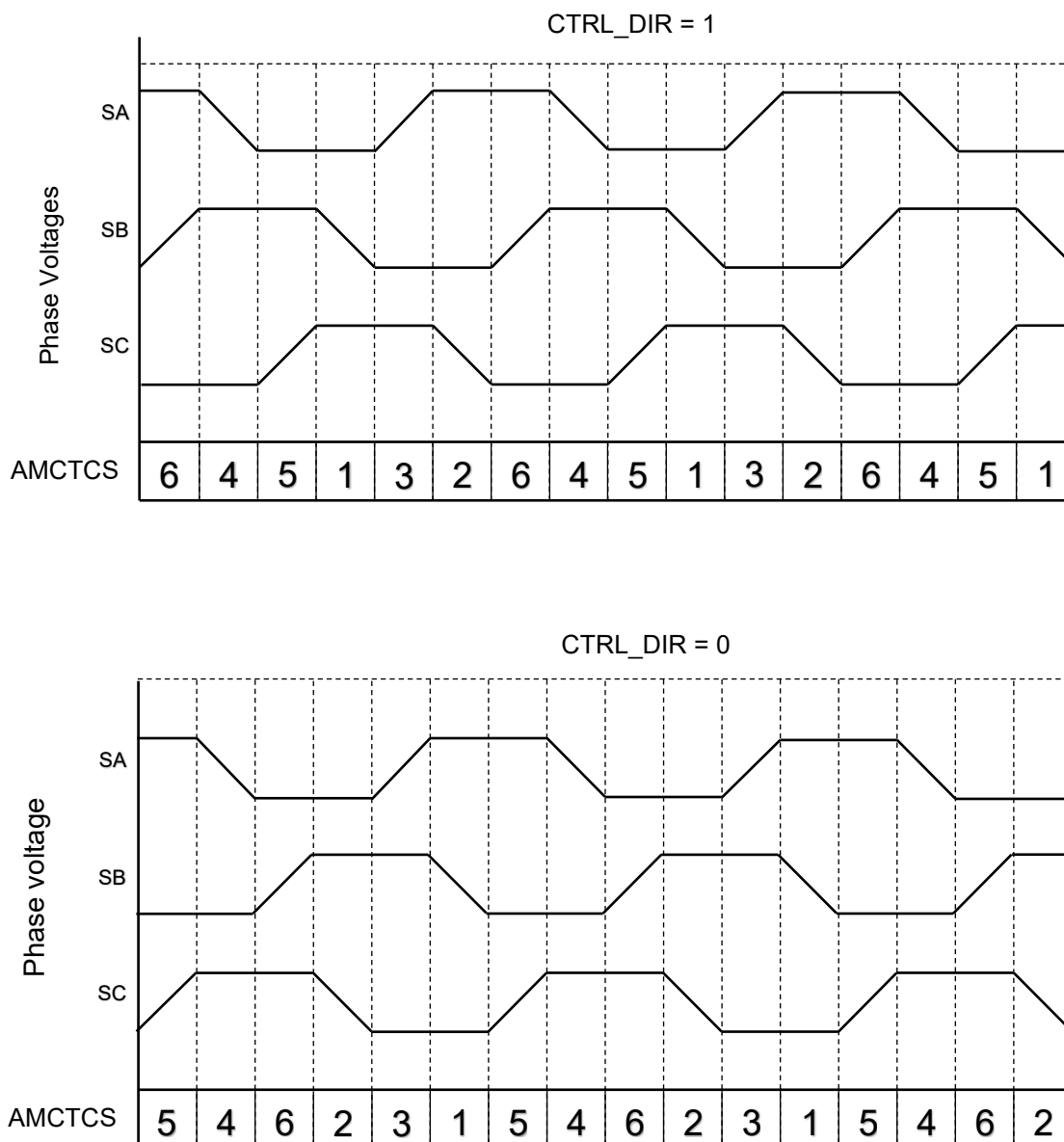


Figure 1-14: BLDC Motor Three Phase Commutation according to Commutation Section AMCTCT

Automotive SoC Motor Driver

1.7 Initial Position Detection (IPD)

The IPD sub-block doesn't provide the initial position of the motor directly, however, it provides the hardware required to detect initial position. The motor phase inductance can vary depending on the rotor position. This is especially true for motors with a high saliency. When two motor phases are driven with PWM signals, it is possible to compare the phase inductances of the driven phases by observing the BEMF comparator output of the third phase (undriven phase). IPD block facilitates recording BEMF comparator outputs over a PWM period and provide information about dominant state of BEMF comparator over the PWM cycle. The final BEMF results can be read within CPU to measure relative phase to phase inductances. The rotor position can ultimately be identified after driving the motor phases in certain sequence such that a comparative result between all three motor phases are obtained.

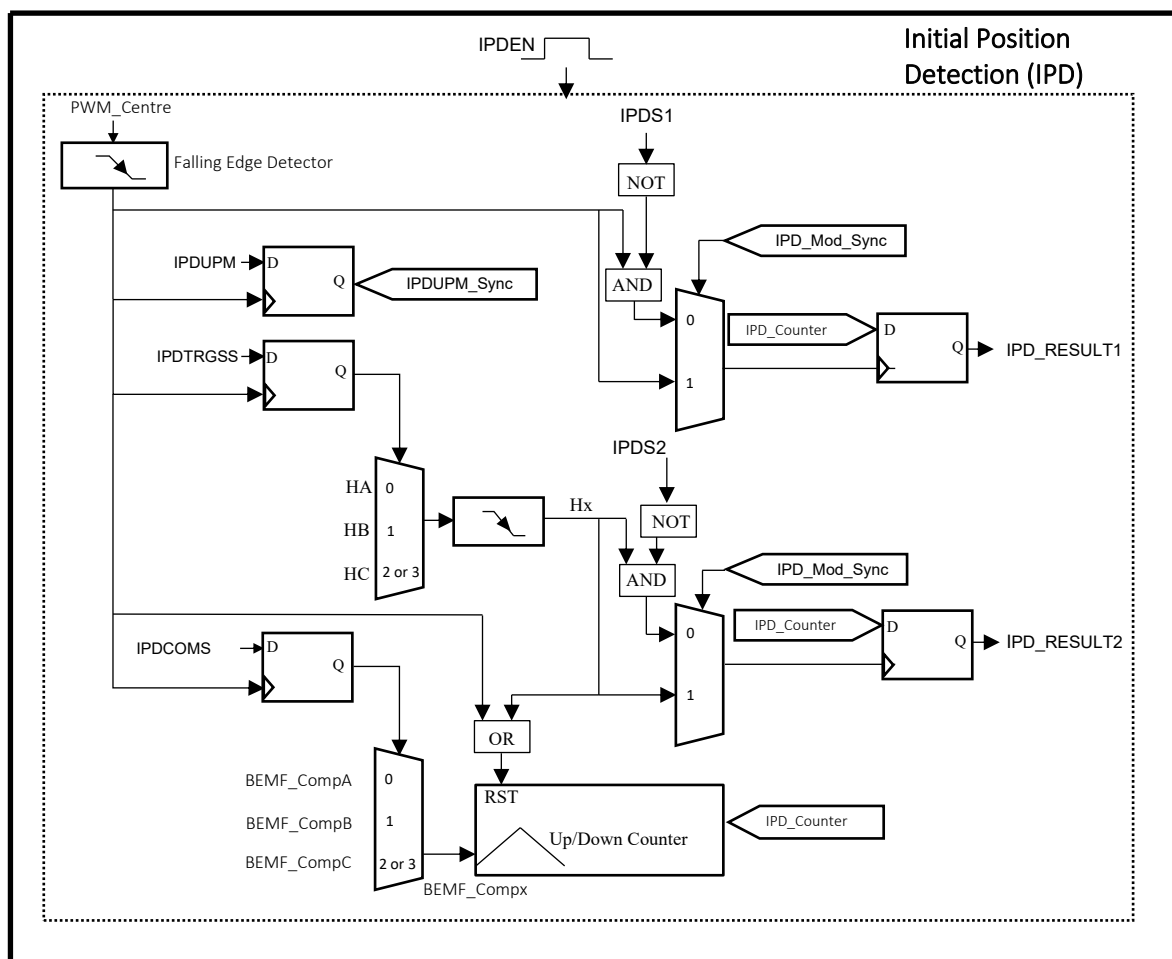


Figure 1-15: Initial Position Detection Block Diagram

Automotive SoC Motor Driver

Figure 1-15 shows the block diagram of the IPD block. The block is enabled when IPDEN is set to 1. When IPDEN is set to zero, IPD_RESULT1 and IPD_RESULT2 are reset to 2^{15} . The inputs IPDCOMS, IPDTRGSS and IPDUPM are synchronized with the falling edge of PWM_Centre and are only update once per falling edge of PWM_Centre. The IPDCOMS, selects one of the three BEMF comparators that will be input to the up/down counter. IPDTRGSS will select the trigger source signal which can be either HA, HB or HC generated by the PWMU. The Up/Down counter is increment by 1 when BEMF_CompX is '1' and it is decremented when BEMF_CompX is '0' at a f_{AMCT} (40Mhz) rate. The result of the counter is saved in IPD_Counter. The IPD_Counter is reset to 2^{15} at the falling edge of Hx signal or falling edge of PWM_Centre or when IPDEN is set to 0. At the falling edge of PWM_Centre, IPDS1 is flagged in the AMCT_STATUS register. This flag can only be cleared by succesful write of 1 to IPDS1 bit or setting IPDEN bit to 0. Similarly, at the falling edge of Hx, IPDS2 is flagged in the AMCT_STATUS register. This flag can only be cleared by succesful write of 1 to IPDS2 bit or setting IPDEN bit to 0. When IPDUPM is set to 1, IPD_RESULT1 is updated with IPD_Counter value at the falling edge of PWM_Centre and IPD_RESULT2 is updated at the falling edge of Hx signal. When IPDUPM is set to 0, IPD_RESULT1 is updated at the falling edge of PWM_Centre only if IPDS1 is cleared. If IPDS1 is not cleared, previous IPD_RESULT1 value is maintained. Similarly, when IPDUPM is set to 0, IPD_RESULT2 is updated at the falling edge of PWM_Centre only if IPDS2 is cleared. If IPDS2 is not cleared, IPD_RESULT2 holds its previous value.

The waveforms shown in Figure 1-16 demonstrates the functional behavior of the IPD Block. In the first PWM cycle IPDTRGSS is set to 0. This means the HA signal is used as a trigger source. IPDCOMS is set to either '2' or '3' which means BEMF_CompC is used as an input to the up/down counter. The IPDUPM is set to 1 which means the IPD_RESULT1 and IPD_RESULT2 are updated regardless of the status bits IPDS1 and IPDS2. The IPD_Counter is incremented initially because BEMF_CompC is logic high. However, at time T_1 , as the BEMF_CompC changes to logic low the IPD_Counter starts to decrement. At time T_2 , the negative edge of the trigger source signal HA occurs, and therefore the result of the IPD_Counter with a value of 'A' with respect to reset value (2^{15}) is recorded in the IPD_RESULT2 register. IPDS2 is also set to indicate the falling edge of signal HA has occurred and measurement is ready. Once the result recorded, IPD_Counter is automatically reset to a default value 2^{15} . In this scenario since the IPD_RESULT2 shows a value greater than 2^{15} , it indicates that the bemf_compC logic high has been a dominant state throughout the measured period. The same procedures follows from time T_2 to T_3 , except this time, the result of the IPD_Counter are recorded in register IPD_RESULT1 at the falling edge of PWM_Centre. IPDS1 is set high to indicate the falling edge of PWM_Centre has occurred and a new measurement for IPD_RESULT1 is ready. In this case, logic low is a dominant state for the BEMF_CompC and therefore the IPD_RESULT1 is update with a value 'D' lower than the default value (2^{15}). The same procedures follows for the other two PWM cycles except that the IPDTRGSS and IPDCOMS are changed to illustrate the block behavior for the other signals. Two important points to notice in the waveform is that at time T_4 , IPD_RESULT1 is updated even though status bit IPDS1 is not cleared. This is because IPDUPM is set to '1' at this time. However, At time T_5 , previous IPD_RESULT1 value is mainted despite the occurance of the falling edge of PWM_Centre. This is because at this point, IPDUPM is set to '0' and IPDS1 is

Automotive SoC Motor Driver

not cleared. IPDS1 and IPDS2 can be cleared by writing '1' to corresponding bits in the AMCT_STATUS register.

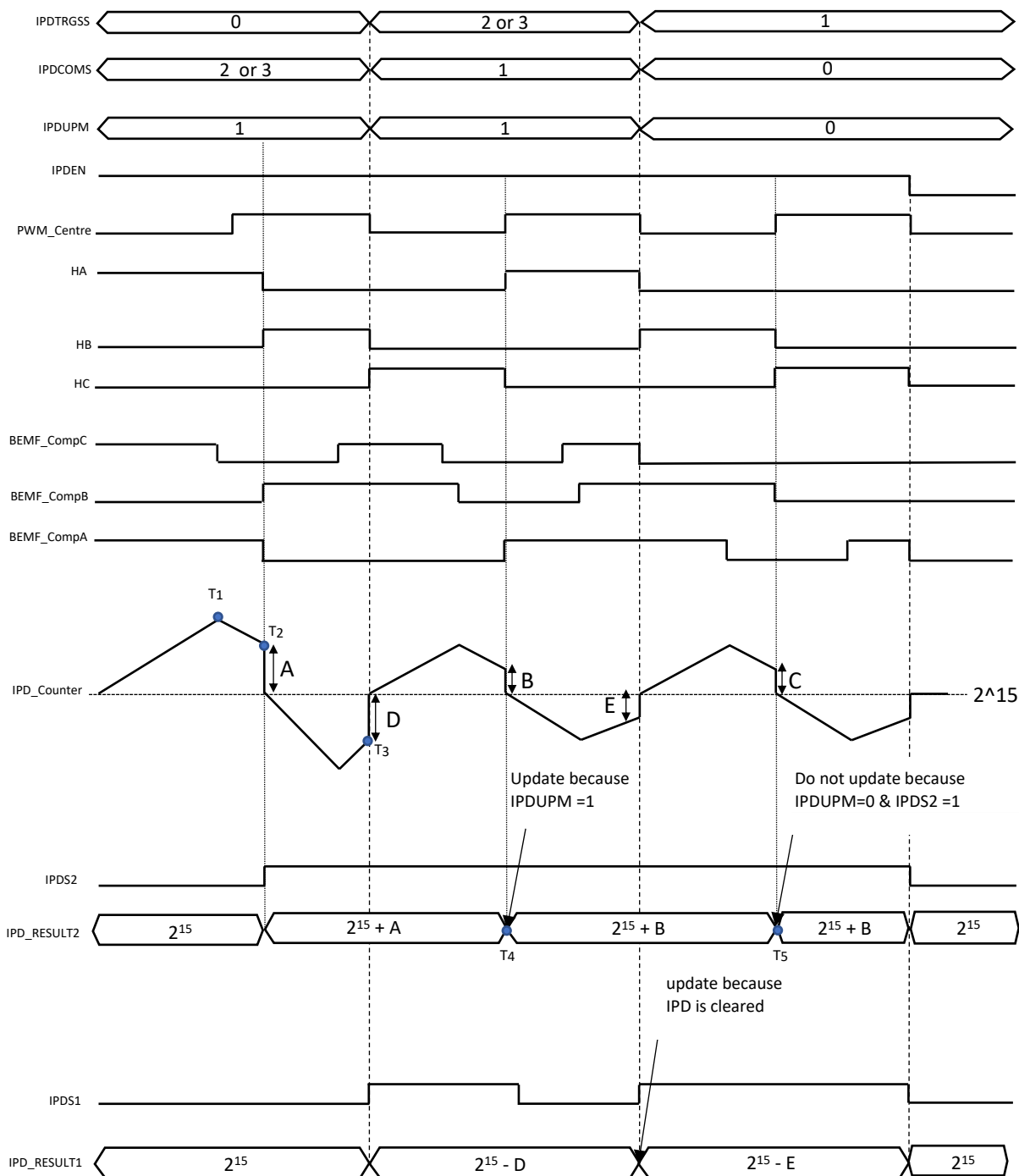


Figure 1-16: Initial Position Detection (IPD) functional behavior

Automotive SoC Motor Driver

The duration difference between a logic high and a logic low for a corresponding BEMF_Comp_x for a measured period can be calculated as below:

From H_x falling edge to PWM_Centre falling edge

$$BEMF_{COMPx}(\text{logic high and low low duration difference}) = \frac{IPD_RESULT1 - 2^{15}}{f_{AMCT}} (s)$$

From PWM_Centre falling edge to H_x Falling edge

$$BEMF_{COMPx}(\text{logic high and logic low duration difference}) = \frac{IPD_RESULT2 - 2^{15}}{f_{AMCT}} (s)$$

Because the counters counts up for a logic high and counts down for logic low, it is clear that if IPD_RESULT1 or IPD_RESULT2 are values greater than 2¹⁵, logic high has been a dominant state for the measured period and vice versa.

For example, at time T₃ if 'A' represents 4000, then IPD_RESULT2 is '2¹⁵+4000'. This means BEMF_CompC dominant state is logic high and the duration difference between logic high and logic low for the measured period, assuming the default f_{AMCT} of 40Mhz, is:

$$\frac{(4000 + 2^{15}) - 2^{15}}{40 \times 10^6} = 0.1 \text{ ms}$$

Automotive SoC Motor Driver

1.8 Current Polarity Detection (CPD)

Current polarity detection block helps identifying the polarity of the motor phase current. One of the advantages of knowing the current polarity is the ability to implement dead-time compensation. Dead-time is required to avoid short circuit in the power bridge FETs, however, the application of dead-time results in distortion in voltage waveform and increase total harmonic distortion (THD). Dead-time compensation can therefore help improving THD. The block diagram of current polarity detection is shown in Figure 1-17.

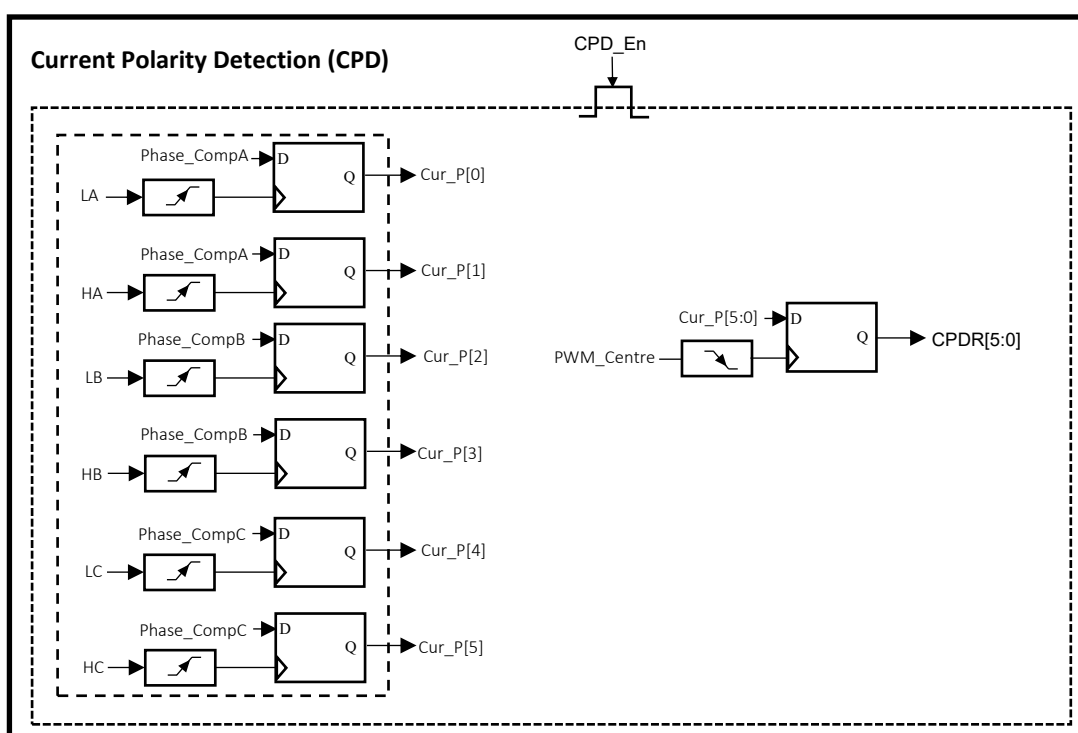


Figure 1-17: Current Polarity Detection (CPD) Block Diagram

Let's consider the current going into the motor phase is positive and the current coming out of a phase is negative. In this case, the phase comparator is set to 0 if the current is positive and it is set to 1 when the current is negative during dead-time. Therefore, by having the knowledge of phase comparator output during each dead time, it is possible to identify the polarity of the current in each phase. In Figure 1-17, the phase comparators are captured at the rising edge of LA, HA, etc. All the results are then synchronized and captured at the falling edge of the PWM_Centre signal and recorded in CPDR in the AMCT_CPD_RESULT register. For example, when CPDR represents the binary number 001011, the phase C current is positive, the phase A current is negative and the current in phase B has just changed polarity from negative to positive. It should be noted the block is enabled when CPDEN is set to 1. When CPDEN is set to 0 CPDR value is reset to 0. The timing diagram for the current polarity detection (CPD) is shown in Figure 1-18, showing when the phase comparators are captured and the synchronization of the results with PWM_Centre.

Automotive SoC Motor Driver

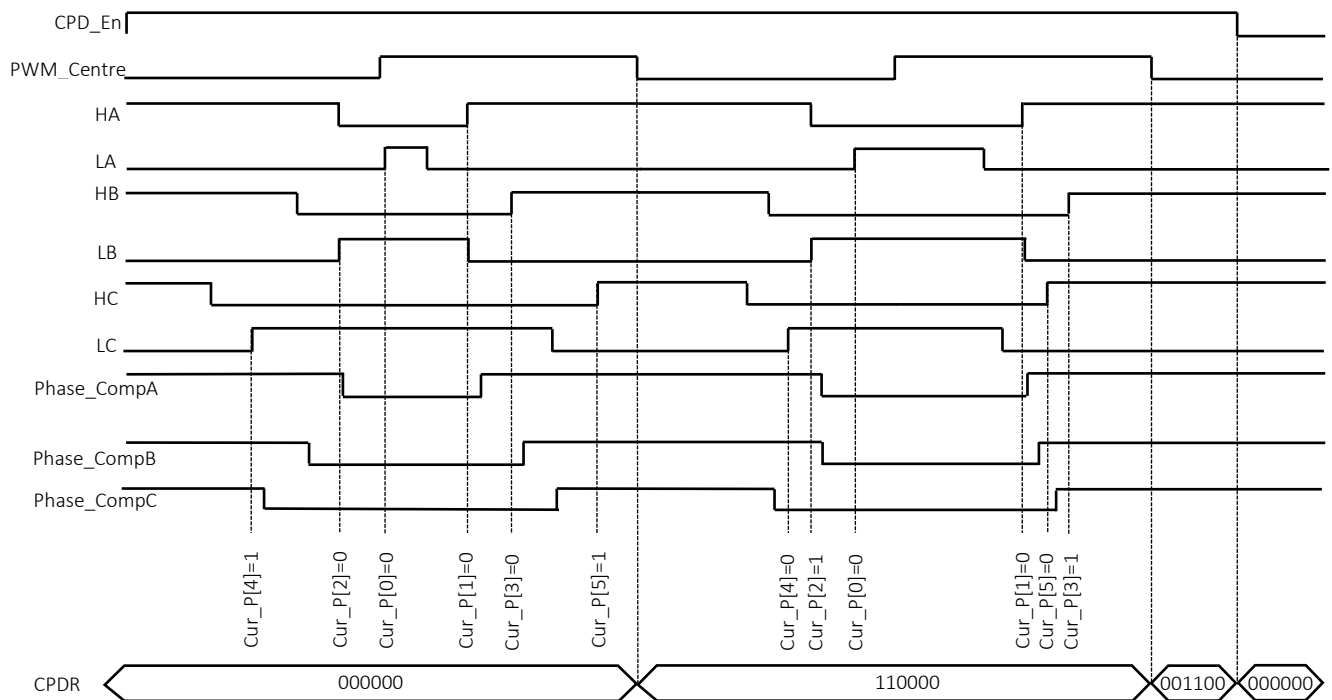


Figure 1-18: Current Polarity Detection (CPD) Timing Diagram

Automotive SoC Motor Driver

1.9 Interrupt Management System

Figure 1-19 shows the block diagram of the interrupt management system. It is possible to generate an interrupt signal, AMCT_IRQ, connected to NVIC, from the status signals available in AMCT_STATUS register. The description for each of the status signals are provided in details in the previous sections. In order to generate interrupt, the interrupt enable bit for a corresponding status bit must be enabled in the AMCT_STATUS_INTEN register. The status information in AMCT_STATUS registers can be cleared by successful write of '1' to a corresponding bit in the AMCT_STATUS register or setting MASTEREN bit to '1'.

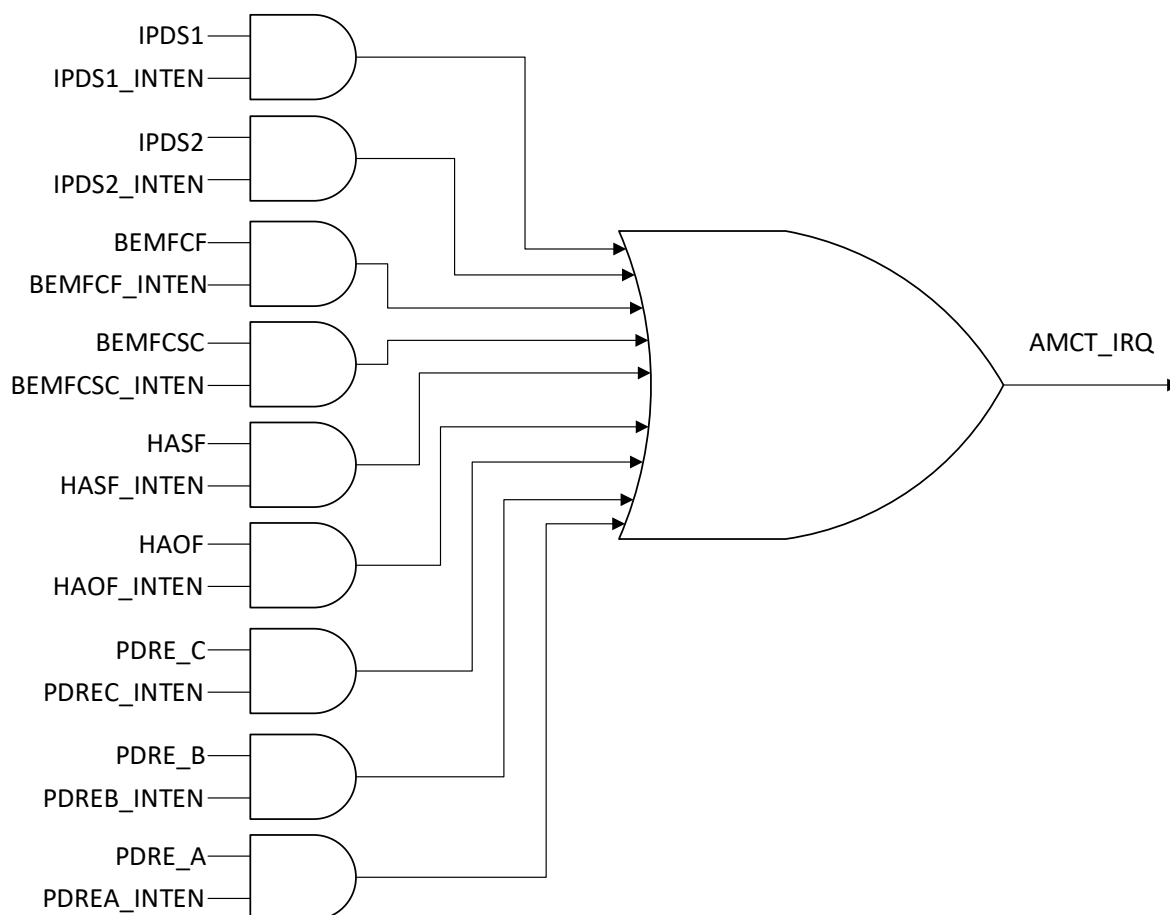


Figure 1-19: Interrupt Management System Block Diagram

Automotive SoC Motor Driver

1.10 Register sets

The AMCT module contains 24 registers in total. The register names, address offsets and access type and reset values are shown in Table 1-4.

AMCT Unit Base Address: 0x40020000

Table 1-4: AMCT Unit Register Map

Register Name	Address Offset	Access Type	Reset Value
AMCT_CONFIG	0x00	R/W	0x00000000
AMCT_COMMAND	0x04	R/W	0x00000000
AMCT_PHASE_DISABLE_CONTROL	0x08	R/W	0x00000000
AMCT_HALL_FILTER_TIME	0x0C	R/W	0x00000000
AMCT_HALL_PHASE_SHIFT	0x10	R/W	0x00000000
AMCT_ANGLE_STEP	0x14	R/W	0x00000000
AMCT_HALL_FREQUENCY_LIMIT	0x18	R/W	0x00000000
AMCT_APPLY_MOTOR_DRIVING_ANGLE_CORRECTION	0x1C	R/W	0x00000000
AMCT_MOTOR_DRIVING_ANGLE_CORRECTION_FACTOR	0x20	R/W	0x00000000
AMCT_INITIAL_DRIVING_ANGLE	0x24	R/W	0x00000000
AMCT_BEMF_DEGAUSS_FILTER_BLANK_TIME	0x28	R/W	0x00000028
AMCT_BEMF_FILTER_TIME	0x2C	R/W	0x00000000
AMCT_INTEN	0x30	R/W	0x00000000
AMCT_PHASE_DISABLE_OUTPUT	0x34	R	0x00000000
AMCT_FILTERED_BEMF_COMPARATOR_OUTPUT	0x38	R	0x00000000
AMCT_BEMF_COMPARATOR_TACHO	0x3C	R	0x00000000
AMCT_STATUS	0x40	R/1C	0x00000000
AMCT_COMMUTATION_SECTION	0x44	R	0x00000000
AMCT_HALL_FREQUENCY	0x48	R	0x00000001
AMCT_MOTOR_DRIVING_ANGLE	0x4C	R	0x00000000
AMCT_BEMF_ANGLE_ERROR	0x50	R	0x00000000
AMCT_IPD_RESULT1	0x54	R	0x00008000
AMCT_IPD_RESULT2	0x58	R	0x00008000
AMCT_CPD_RESULT	0x5C	R	0x00000000

Automotive SoC Motor Driver

AMCT_CONFIG

Reset Value: 0x00000000

Register Address: 0x40020000

This register defines the configuration settings needed for operation of the sub-systems within AMCT.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	IPDUPM	IPDCOMS[1]	IPDCOMS[0]	IPDTRGS S[1]	IPDTRGS S[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	HACS[2]	HACS[1]	HACS[0]	HABS[2]	HABS[1]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
HABS[0]	HAAS[2]	HAAS[1]	HAAS[0]	PDMS	DFB	CTRL_DIR	AMCTM
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
AMCTM	0	AMCT Mode Defines the mode of operation from which the “AMCT Phase Disable Output” and “AMCT Commutation Section” registers are updated. 0: BEMF-based based mode of operation 1: Hall-based mode of operation
CTRL_DIR	1	Motor Control Direction 0: Reverse 1: Forward
DFB	2	Degaus Filter Bypass: 0: Degaus filtering is not bypassed 1: Degaus filtering is bypassed
PDMS	3	Phase Disable Selection: 0: Phase disable command is generated by AMCT module algorithm

Automotive SoC Motor Driver

1: Phase disable command is generated by configuring "AMCT_PHASE_DISABLE_CONTROL" within software

HAAS	6:4	Hall A GPI Selection 0: PD0 1: PD1 2: PD2 ... 7: PD7
HABS	9:7	HALL B GPI Selection 0: PD0 1: PD1 2: PD2 ... 7: PD7
HACS	12:10	HALL C GPI Selection 0: PD0 1: PD1 2: PD2 ... 7: PD7
-	15:13	Reserved
IPDTRGSS	17:16	IPD Trigger Source Selection 0: GHA 1: GHB 2 or 3: GHC
IPDCOMS	19:18	IPD Comparator Selection 0: Phase A Comparator 1: Phase B Comparator 2 or 3: Phase C comparator
IPDUPM	20	IPD Update Mode 0: Update the result registers IPD_RESULT1 and IPD_RESULT2 whenever new value is available 1: Update the result registers IPD_RESULT1 only if IPDS1 is cleared and update IPD_RESULT2 only if IPDS2 is cleared

Automotive SoC Motor Driver

AMCT_COMMAND

Register Address: 0x40020004

Reset Value: 0x00000000

This register enables or disables the functionalities of different sub-systems within AMCT unit.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Res	Res	Res	BEMFEN	HAEN	CPDEN	IPDEN	MASTEREN
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
MASTEREN	0	Master Enable 0: Disable the whole AMCT block 1: Enable the whole AMCT block
IPDEN	1	Initial Position Detection Enable 0: Disable IPD subsystem 1: Enable IPD subsystem
CPDEN	2	Current Polarity Detection Enable 0: Disable CPD subsystem 1: Enable CPD subsystem
HAEN	3	Hall Enable 0: Disable hall-based subsystem 1: Enable hall-based subsystem
BEMFEN	4	BEMF Enable 0: Disable BEMF-based subsystem 1: Enable BEMF based subsystem

Automotive SoC Motor Driver

AMCT_PHASE_DISABLE_CONTROL

Register Address: 0x40020008

Reset Value: 0x00000000

This register defines the final phase disable output if PDS is set to '1' in AMCT_CONFIG register.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Res	Res	Res	Res	Res	MIP_DIS		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

MP_DIS	2:0	Manual Phase Disable Control.
---------------	-----	--------------------------------------

If (PDS =1): FPHASE_Dis = MIP_DIS

Automotive SoC Motor Driver

AMCT_HALL_FILTER_TIME

Reset Value: 0x00000000

Register Address: 0x4002000C

This register defines the hall-based system debounce filter time.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
HAFT[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
HAFT[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

HAFT	15:0	Hall Filter Time.
-------------	------	--------------------------

This parameter defines the debounce filter time for Hall_based system.

$$t_{HFT} = \frac{HAFT}{f_{AMCT}} \text{ s}$$

Where f_{AMCT} represent the clock frequency in AMCT module and by default set to 40Mhz.

Automotive SoC Motor Driver

AMCT_HALL_PHASE_SHIFT

Reset Value: 0x00000000

Register Address: 0x40020010

This register defines the phase shift or phase advance generated for the combination hall sensor inputs.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
HAPS[31:24]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
HAPS[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
HAPS[15:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
HAPS[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

HAPS 31:0 (signed)

Hall Phase Shift.

This signed parameter defines the amount of phase shift in the Hall Commutation Section (HACS). To achieve phase advance, HAPS should be negative and to achieve phase delay HAPS should be positive. When HAPS is set to 0, no phase shift is generated. The amount of advance or delay angle can be calculated based on below equations:

Phase Advance Angle:

$$\text{Phase Advance} = \frac{(HAF + HAPS)}{HAF} \times 60 \text{ degree}$$

Phase Delay Angle:

$$\text{Phase Delay} = \frac{HAPS}{HAF} \times 60 \text{ degree}$$

The maximum amount of phase advance or phase delay shouldn't be greater than 60 degree.

Automotive SoC Motor Driver

AMCT_ANGLE_STEP

Reset Value: 0x00000000

Register Address: 0x40020014

This register will set the rate at which the motor driving angle, MDA, changes.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
HALF[31:24]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
HALF[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
HALF[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
HALF[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

MDAS	31:0	Motor Driving Angle Step
-------------	------	---------------------------------

This parameter defines the rate at which the motor driving angle changes at a rate of f_{AMCT} and is related to motor electrical frequency as below:

$$MDAS = \frac{f_{Motor-Electrical}}{f_{AMCT}} \times 2^{32}$$

where:

$f_{Motor-Electrical}$ = motor electrical frequency in Hz decided by control algorithm

f_{amct} = AMCT module clock frequency which by default is set to 40Mhz

Automotive SoC Motor Driver

AMCT_HALL_FREQUENCY_LIMIT

Reset Value: 0x00000000

Register Address: 0x40020018

This register will set the minimum frequency of Hall sensor inputs that can be measured by Hall-based system.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
HALF[31:24]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
HALF[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
HALF[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
HALF[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

HAFL	31:0	Hall Frequency Limit
-------------	------	-----------------------------

This parameter will set the limit for the minimum rate of change (frequency) in Hall_State that can be captured. If the Hall_State rate of change is slower than defined by below equation, the overflow flag HAOF will be set in AMCT_STATUS register and HAF will reset to 1.

$$\text{Hall_State (minimum rate of change)} = \frac{f_{AMCT}}{HAFL} \text{ Hz}$$

Where f_{AMCT} represent the clock frequency in AMCT module and by default set to 40Mhz.

Automotive SoC Motor Driver

AMCT_Apply_Motor_Driving_Angle_Correction

Register Address: 0x4002001C

Reset Value: 0x00000000

On the rising edge of AMDAC parameter in this register, the angle correction factor MDAC will be applied to motor driving angle, MDA.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Res	Res	Res	Res	Res	Res	Res	AMDAC
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

AMDAC	0	Apply Motor Driving Angle Correction
--------------	---	---

Transition from 0 to 1 (rising edge): Apply motor driving angle correction (MDAC) to the motor driving angle (MDA) in the BEMF based system.

Automotive SoC Motor Driver

AMCT_Motor_Driving_Angle_Correction_Factor

Register Address: 0x40020020

Reset Value: 0x00000000

This register defines the correction factor that can be applied to motor driving angle MDA if the motor phase commutations are not correct.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
MDAC[31:24]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
MDAC[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
MDAC[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
MDAC[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

MDAC	31:0(signed)
-------------	--------------

MDAC

This signed parameter is a correction factor that can be added or subtracted from MDA to improve motor commutation point. The addition or subtraction depends on the CTRL_DIR and the sign of MDAC parameter. Refer to motor driving angle generator [section](#) for more information.

Note: This parameter is only added to or subtracted from MDA on the 0 to 1 transition (rising edge) of AMDAC

Automotive SoC Motor Driver

AMCT_INITIAL_DRIVING_ANGLE

Reset Value: 0x00000000

Register Address: 0x40020024

This register defines the initial value for the motor driving angle MDA.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
MIDA[31:24]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
MIDA[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
MIDA[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
MIDA[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

MIDA	31:0	Motor Initial Driving Angle
-------------	------	------------------------------------

When BEMF-based system is just enabled (BEMFEN transition from 0 to 1), this parameter defines the initial value of the motor driving angle MDA and is related to angle in degree as below:

$$Angle_{initial} = \frac{MIDA}{2^{32} - 1} \times 360 \text{ degree}$$

Automotive SoC Motor Driver

AMCT_BEMF_DEGAUSS_FILTER_BLANK_TIME

Register Address: 0x40020028

Reset Value: 0x00000028

This register defines the blanking time for the BEMF comparator degauss filter.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	BEMFDFBT[9:8]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
BEMFDFBT[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

BEMFDFBT	9:0	BEMF Degauss Filter Blank Time
-----------------	------------	---------------------------------------

This parameter defines the blanking time for the degauss filter in the BEMF-based system.

$$\text{Degauss Filter Blank Time} = \frac{\text{BEMFDFBT}}{f_{AMCT}} s$$

Where f_{AMCT} represent the clock frequency in AMCT module and by default set to 40Mhz.

Automotive SoC Motor Driver

AMCT_BEMF_FILTER_TIME

Reset Value: 0x00000000

Register Address: 0x4002002C

This register defines the debounce filter time for the BEMF comparators output.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	BEMFFT[19:16]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
BEMFFT[15:8]							
R/W	R/W	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
BEMFFT[7:0]							
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

BEMFFT	19:0	BEMF Filter Time
---------------	------	-------------------------

This parameter defines the debounce filter time for the bemf comparators output.

$$\text{debounce filter time} = \frac{\text{BEMFFT}}{f_{AMCT}} \text{ s}$$

Where f_{AMCT} represent the clock frequency in AMCT module and by default set to 40Mhz.

Automotive SoC Motor Driver

AMCT_INTEN

Reset Value: 0x00000000

Register Address: 0x40020030

This register enables the interrupt generation for signals in AMCT_STATUS register.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	Res	IPDS1_INTEN
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IPDS2_INTEN	BEMFCF_INTEN	BEMFCSC_INTEN	HASF_INTEN	HAOF_INTEN	PDRE_C_INTEN	PDRE_B_INTEN	PDRE_A_INTEN
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
IPDS1_INTEN	8	IPD STATUS 1 Interrupt Enable 0: Disable interrupt generation for IPDS1 1: Enable interrupt generation for IPDS1
IPDS2_INTEN	7	IPD STATUS 2 Interrupt Enable 0: Disable interrupt generation for IPDS2 1: Enable interrupt generation for IPDS2
BEMFCF_INTEN	6	BEMF Comparator Fault Interrupt Enable 0: Disable interrupt generation for BEMFCF 1: Enable interrupt generation for BEMFCF
BEMFCSC_INTEN	5	BEMF Comparator State Change Interrupt Enable 0: Disable interrupt generation for BEMFCSC 1: Enable interrupt generation for BEMFCSC

Automotive SoC Motor Driver

HASF_INTEN	4	Hall State Fault Interrupt Enable 0: Disable interrupt generation for HASF 1: Enable interrupt generation for HASF
HAOF_INTEN	3	HALL Overflow Interrupt Enable 0: Disable interrupt generation for HAOF 1: Enable interrupt generation for HAOF
PDRE_C_INTEN	2	Phase C disable Rising Edge Interrupt Enable 0: Disable interrupt generation for PDRE_C 1: Enable interrupt generation for PDRE_C
PDRE_B_INTEN	1	Phase B disable Rising Edge Interrupt Enable 0: Disable interrupt generation for PDRE_B 1: Enable interrupt generation for PDRE_B
PDRE_A_INTEN	0	Phase A disable Rising Edge Interrupt Enable 0: Disable interrupt generation for PDRE_A 1: Enable interrupt generation for PDRE_A

Automotive SoC Motor Driver

AMCT_PHASE_DISABLE_OUTPUT

Reset Value: 0x00000000

Register Address: 0x40020034

This registers provides the status of the phase disable output signal, FPHASE_DIS.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Res	Res	Res	Res	Res	FPHASE_DIS[2:0]		
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
FPHASE_DIS	2:0	Final Phase Disable output for Phases A, B and C

Bit 2:

0: PhaseC_Dis is low

1: PhaseC_Dis is high

Bit 1:

0: PhaseB_Dis is low

1: PhaseB_Dis is high

Bit 0:

0: PhaseA_Dis is low

1: PhaseA_Dis is high

Automotive SoC Motor Driver

AMC_FILTERED_BEMF_COMPARATOR_OUTPUT

Reset Value: 0x00000000

Register Address: 0x40020038

This register provides the state of the BEMF comparators after filtering stages.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Res	Res	Res	Res	Res	FBEMFCO[2:0]		
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
FBEMFCO	2:0	Filtered BEMF Comparator Output

Bit 2:

0: BEMF_ComparatorC_Filt is low

1: BEMF_ComparatorC_Filt is high

Bit 1:

0: BEMF_ComparatorB_Filt is low

1: BEMF_ComparatorB_Filt is high

Bit 0:

0: BEMF_ComparatorA_Filt is low

1: BEMF_ComparatorA_Filt is high

Automotive SoC Motor Driver

AMCT_BEMF_COMPARATOR_TACHO

Reset Value: 0x00000000

Register Address: 0x4002003C

This register provides the state of the BEMF_Comp_Tacho signal

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Res	Res	Res	Res	Res	Res	Res	BEMFCT
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

BEMFCT	0	BEMF Comparator Tacho
---------------	----------	------------------------------

This is the same signal as BEMF_Comp_Tacho and is toggled each time there is a change in FBEMFCO.

0: Bemf_Comp Tacho is low

1: BEMF_Comp_Tacho is high

Note: This signal is also available as a source output for GPIO pins (refer to GPIO documentation for more information).

Automotive SoC Motor Driver

AMCT_STATUS

Register Address: 0x40020040

Reset Value: 0x00000000

This register provides the status information for the important parameters in AMCT. The status information in this register can be cleared by writing '1' to a corresponding bit or setting the master enable bit, MASTEREN, to '0'.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R/1C	R/1C	R/1C	R/1C	R/1C	R/1C	R/1C	R/1C

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R/1C	R/1C	R/1C	R/1C	R/1C	R/1C	R/1C	R/1C

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	Res	IPDS1
R/1C	R/1C	R/1C	R/1C	R/1C	R/1C	R/1C	R/1C

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IPDS2	BEMFCF	BEMFCSC	HASF	HAOF	PDRE_C	PDRE_B	PDRE_A
R/1C	R/1C	R/1C	R/1C	R/1C	R/1C	R/1C	R/1C

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

IPDS1	8	IPD STATUS 1 ¹ 0: IPD_RESULT1 information is not ready 1: IPD_RESULT1 information is ready
IPDS2	7	IPD STATUS 2 ¹ 0: IPD_RESULT2 information is not ready 1: IPD_RESULT2 information is ready
BEMFCF	6	BEMF Comparator Fault ^{2,3} 0: There is no fault state in the FBEMFCO 1: There is a fault state in the FBEMFCO
BEMFCSC	5	BEMF Comparator State Change ³ 0: No Change has occurred to the value of FBEMFCO 1: A change has occurred to FBEMFCO
HASF	4	Hall State Fault ^{4,5} 0: There is no fault value in the Hall_State 1: There is a fault value in the Hall_State

Automotive SoC Motor Driver

HAOF	3	HALL Overflow ⁵ 0: Hall_State Frequency is not below the threshold set by HAFL and overflow not detected 1: Hall_State Frequency is below the threshold set by HAFL and overflow detected
PDRE_C	2	Phase C disable Rising Edge ⁶ 0: Rising Edge event not detected for signal PDRE_C 1: Rising Edge event detected for signal PDRE_C
PDRE_B	1	Phase B disable Rising Edge ⁶ 0: Rising Edge event not detected for signal PDRE_C 1: Rising Edge event detected for signal PDRE_C
PDRE_A	0	Phase A disable Rising Edge ⁶ 0: Rising Edge event not detected for signal PDRE_C 1: Rising Edge event detected for signal PDRE_C

Notes:

- 1 - The status bit can be cleared by write of '1' to the corresponding bit or setting MASTEREN or IPDEN to '0'.
- 2 - States 0 and 7 are considered fault state for FBEMFCO[2:0].
- 3 - The status bits can be cleared by write of '1' to the corresponding bit or setting MASTEREN or BEMFEN to '0'.
- 4 - States 0 and 7 are considered fault state for Hall_State.
- 5 - The status bits can be cleared by write of '1' to the corresponding bit or setting MASTEREN or HAEN to '0'.
- 6 - The status bits can be cleared by write of '1' to the corresponding bit or setting MASTEREN to '0'.

Automotive SoC Motor Driver

AMCT_COMMUTATION_SECTION

Reset Value: 0x00000000

Register Address: 0x40020044

This register provides the AMCTCS generated by BEMF-based system or Hall-Based system depending on the AMCTM. This parameter can be used to energize motor phases in a correct sequence.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Res	Res	Res	Res	Res	AMCTCS2	AMCTCS1	AMCTCS0
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

AMCTCS	2:0	AMCT Commutation Section
---------------	-----	---------------------------------

The commutation section report to energize correct motor phases. Refer to output stage control [section](#) for more information.

Automotive SoC Motor Driver

AMCT_HALL_FREQUENCY

Reset Value: 0x00000000

Register Address: 0x40020048

This register provides the rate of change of hall state inputs Hall_State.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
HAF[31:24]							
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
HAF[23:16]							
R	R	R	R	R	R	R	R

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
HAF[15:8]							
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
HAF[7:0]							
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

HAF	31:0	Hall Sensors Frequency
------------	------	-------------------------------

Provides information about the rate of change in the Hall_State.

$$\text{Hall State Rate of Change} = \frac{f_{AMCT}}{HAF - 1} \text{ Hz}$$

Note:

- 1- When there is no change in the hall_state , HAF = 1. Care must be taken when calculating equation above.
- 2- f_{AMCT} is by default set to 40Mhz.

Automotive SoC Motor Driver

AMCT_Motor_DRIVING_ANGLE

Reset Value: 0x00000000

Register Address: 0x4002004C

This register provides the driving angle generated by the angler generator sub-system in the bemf-based system.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
MDA[31:24]							
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
MDA[23:16]							
R	R	R	R	R	R	R	R

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
MDA[15:8]							
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
MDA[7:0]							
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

MDA	31:0	Motor Driving Angle
------------	------	---------------------

This parameter is related to motor driving angle which can be calculated based on the below equation:

$$Driving\ Angle = \frac{MDA}{2^{32} - 1} \times 360\ degree$$

Automotive SoC Motor Driver

AMCT_BEMF_ANGLE_ERROR

Register Address: 0x40020050

Reset Value: 0x00000000

This register provides the motor BEMF angle error with respect to the driving angle (MDA).

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
BEMFAE3 1	BEMFAE3 0	BEMFAE2 9	BEMFAE2 8	BEMFAE2 7	BEMFAE2 6	BEMFAE2 5	BEMFAE2 4
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
BEMFAE2 3	BEMFAE2 2	BEMFAE2 1	BEMFAE2 0	BEMFAE1 9	BEMFAE1 8	BEMFAE1 7	BEMFAE1 6
R	R	R	R	R	R	R	R

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
BEMFAE1 5	BEMFAE1 4	BEMFAE1 3	BEMFAE1 2	BEMFAE1 1	BEMFAE1 0	BEMFAE 9	BEMFAE 8
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
BEMFAE7	BEMFAE6	BEMFAE5	BEMFAE4	BEMFAE3	BEMFAE2	BEMFAE1	BEMFAE0
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
BEMFAE	31:0 (signed)	BEMF Angle Error This parameter defines the amount of electrical angle error between the motor driving angle (BEMFAE) and the actual motor bemf angle. $\text{Angle Error} = \frac{\text{BEMFAE} \times \text{MDAS}}{2^{32} - 1} \times 360 \text{ degree}$ BEMFAE is a signed number. A negative number indicates the motor BEMF angle is lagging the driving motor angle (MDA). A positive number indicates the motor BEMF angle is leading the driving motor angle (MDA).

Automotive SoC Motor Driver

AMCT_IPD_RESULT1

Reset Value: 0x8000

Register Address: 0x40020054

This register provides the duration difference between bemf comparator high and low state captured between falling edge of Hx signal and falling edge of PWM_Centre signal.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8
IPDR1[15:8]							
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IPDR1[7:0]							
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

IPDR1	15:0	Initial Position Detection Result 1
--------------	-------------	--

Represent the duration difference between bemf comparator high state and low state between the falling edge of Hx¹ signal and PWM_Centre². This parameter is recorded at the falling edge of PWM_Centre² signal.

When IPDR1>0x8000, bemf comparator output logic high is a dominant state.

When IPDR1<0x8000, bemf comparator output logic low is dominant state.

The duration difference between logic low and high can be calculated as below:

$$BEMF_{COMPx}(\text{logic high and low duration difference}) = \frac{IPD_RESULT1 - 2^{15}}{f_{AMCT}} (s)$$

Notes:

- 1- Hx: High side signal generated from PWM module (x can represent Phase A, B or C) selected by IPDTRGSS
- 2- PWM_Centre: a signal indicating the start and end of the PWM period generated from PWM module

Automotive SoC Motor Driver

AMCT_IPD_RESULT2

Reset Value: 0x8000

Register Address: 0x40020058

This register provides the duration difference between bemf comparator high and low state captured between falling edge of PWM_Centre signal and falling edge of Hx signal.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8
IPDR2[15:8]							
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IPDR2[7:0]							
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

IPDR1	15:0	Initial Position Detection Result 1
--------------	-------------	--

Represent the duration difference between bemf comparator high state and low state between the falling edge PWM_Centre² and Hx¹ signal. This parameter is recorded at the falling edge of Hx² signal.

When IPDR1>0x8000, bemf comparator output logic high is a dominant state.

When IPDR1<0x8000,bemf comparator output logic low is dominant state.

The duration difference between logic low and high can be calculated as below:

$$BEMF_{COMPx}(\text{logic high and low duration difference}) = \frac{IPD_RESULT1 - 2^{15}}{f_{AMCT}} (s)$$

Notes:

- 1- Hx: High side signal generated from PWM module (x can represent Phase A, B or C) selected by IPDTRGSS
- 2- PWM_Centre: a signal indicating the start and end of the PWM period generated from PWM module

Automotive SoC Motor Driver

AMCT_CPD_RESULT

Register Address: 0x4002005C

Reset Value: 0x8000

This register provides information about motor phase current polarity.

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

Bit15	Bit14	Bit13	Bit12	Bit11	Bit10	Bit9	Bit8
Res	Res	Res	Res	Res	Res	Res	Res
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Res	Res	CPDR[5:0]					
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit, read as 0

Field	Bits	Description
-------	------	-------------

Automotive SoC Motor Driver

CPDR	5:0	Current Polarity Detection Result Represents the polarity of current going into the three phase BLDC motor. This parameter is updated at the falling edge of the PWM_Centre¹. <table border="1" data-bbox="464 488 1385 618"> <thead> <tr> <th>Bit1</th><th>Bit0</th><th>Phase A Current Polarity</th></tr> </thead> <tbody> <tr> <td>0</td><td>0</td><td>positive</td></tr> <tr> <td>1</td><td>0</td><td>under transition²</td></tr> <tr> <td>0</td><td>1</td><td>under transition²</td></tr> <tr> <td>1</td><td>1</td><td>Negative</td></tr> </tbody> </table> <table border="1" data-bbox="464 647 1385 777"> <thead> <tr> <th>Bit3</th><th>Bit2</th><th>Phase B Current</th></tr> </thead> <tbody> <tr> <td>0</td><td>0</td><td>Positive</td></tr> <tr> <td>1</td><td>0</td><td>under transition²</td></tr> <tr> <td>0</td><td>1</td><td>under transition²</td></tr> <tr> <td>1</td><td>1</td><td>negative</td></tr> </tbody> </table> <table border="1" data-bbox="464 806 1385 936"> <thead> <tr> <th>Bit5</th><th>Bit4</th><th>Phase C Current</th></tr> </thead> <tbody> <tr> <td>0</td><td>0</td><td>positive</td></tr> <tr> <td>1</td><td>0</td><td>under transition²</td></tr> <tr> <td>0</td><td>1</td><td>under transition²</td></tr> <tr> <td>1</td><td>1</td><td>negative</td></tr> </tbody> </table> Note: 1- PWM_Centre: a signal indicating the start and the end of the PWM period generated from PWM module 2- The current polarity is transitioning from positive to negative or vice versa. The transition depends on the previous value read. If previous value was read as binary (11), then the transition is from positive to negative. If previous was read as binary (00), then the transition is from negative to positive. positive indicate the current flowing into the motor phase.	Bit1	Bit0	Phase A Current Polarity	0	0	positive	1	0	under transition ²	0	1	under transition ²	1	1	Negative	Bit3	Bit2	Phase B Current	0	0	Positive	1	0	under transition ²	0	1	under transition ²	1	1	negative	Bit5	Bit4	Phase C Current	0	0	positive	1	0	under transition ²	0	1	under transition ²	1	1	negative
Bit1	Bit0	Phase A Current Polarity																																													
0	0	positive																																													
1	0	under transition ²																																													
0	1	under transition ²																																													
1	1	Negative																																													
Bit3	Bit2	Phase B Current																																													
0	0	Positive																																													
1	0	under transition ²																																													
0	1	under transition ²																																													
1	1	negative																																													
Bit5	Bit4	Phase C Current																																													
0	0	positive																																													
1	0	under transition ²																																													
0	1	under transition ²																																													
1	1	negative																																													

Automotive SoC Motor Driver

Revision Control:

July 21/2021	Initial Rev
Nov 11/2021	Updated Logo
May 17/2023	Fixed the issue in graphs caused by previous update. Added description to the title.
July 31/2024	Fixed the typo in CPD_RESULT Register address Fixed the typo in AMCTM setting.
Nov 19/2024	Corrected the config register for parameter CTRL_DIR Corrected the description for parameter BEMFCF
July 11/2025	Changed bit 3 parameter in AMCT_CONFIG register to PDMS Changed parameter name in AMCT_PHASE_DISABLE_CONTROL register to MP_DIS
Feb 20/2025	Corrected bit position for HallC GPIO

Copyright ©2026, Allegro MicroSystems, Inc

Allegro MicroSystems, Inc reserves the right to make, from time to time, such departures from the detail specifications as may be required to permit improvements in the performance, reliability, or manufacturability of its products. Before placing an order, the user is cautioned to verify that the information being relied upon is current.

Allegro's products are not to be used in life support devices or systems, if a failure of an Allegro product can reasonably be expected to cause the failure of that life support device or system, or to affect the safety or effectiveness of that device or system.

The information included herein is believed to be accurate and reliable. However, Allegro MicroSystems, Inc assumes no responsibility for its use; nor for any infringement of patents or other rights of third parties which may result from its use.